Android: Basics on development

Desenvolvimento de Software e Sistemas Móveis (DSSMV)
Licenciatura em Engenharia de Telecomunicações e Informática
LETI/ISEP

2026/27

Paulo Baltarejo Sousa and Carlos Filipe Freitas
{pbs,caf}@isep.ipp.pt



Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

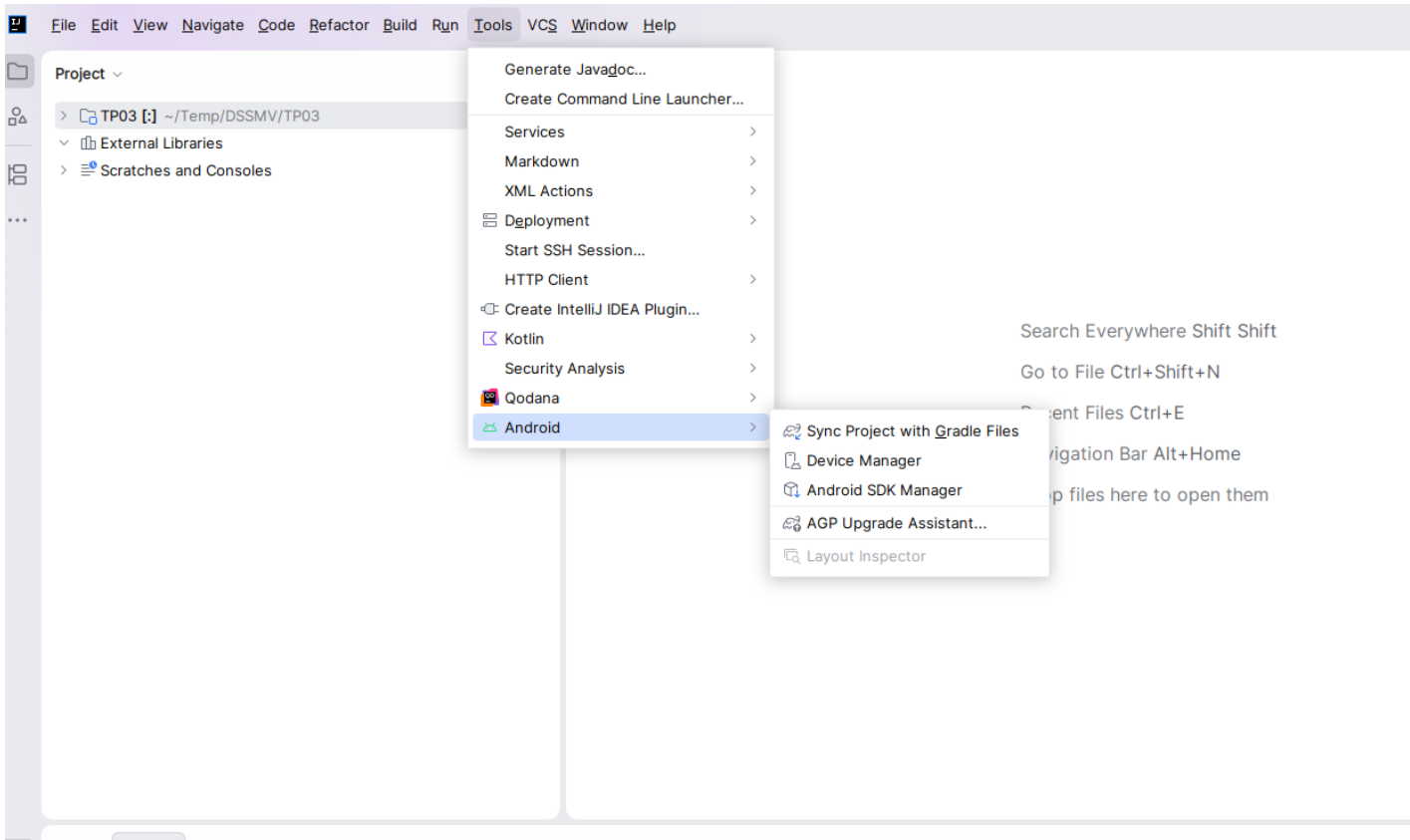
Outline

1. Setting up IntelliJ IDEA
2. IntelliJ IDEA
3. Basics
4. Graphical Interface
5. `AdapterView`

Setting up IntelliJ IDEA

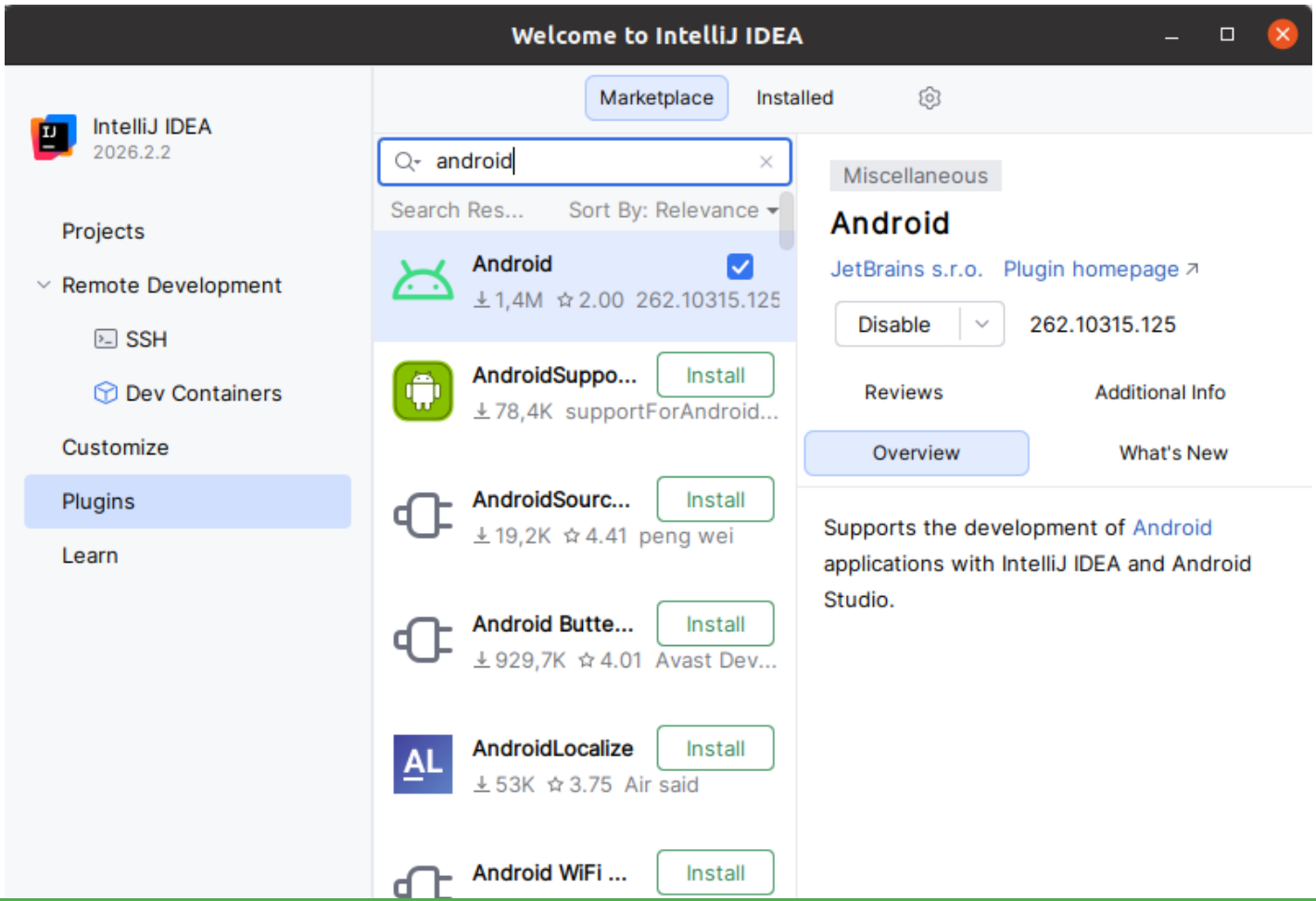
IntelliJ IDEA

- All documentation about IntelliJ IDEA can be found at `https://www.jetbrains.com/help/idea/getting-started.html`.
- All Android Related features are avaialble from `Tools->Android`

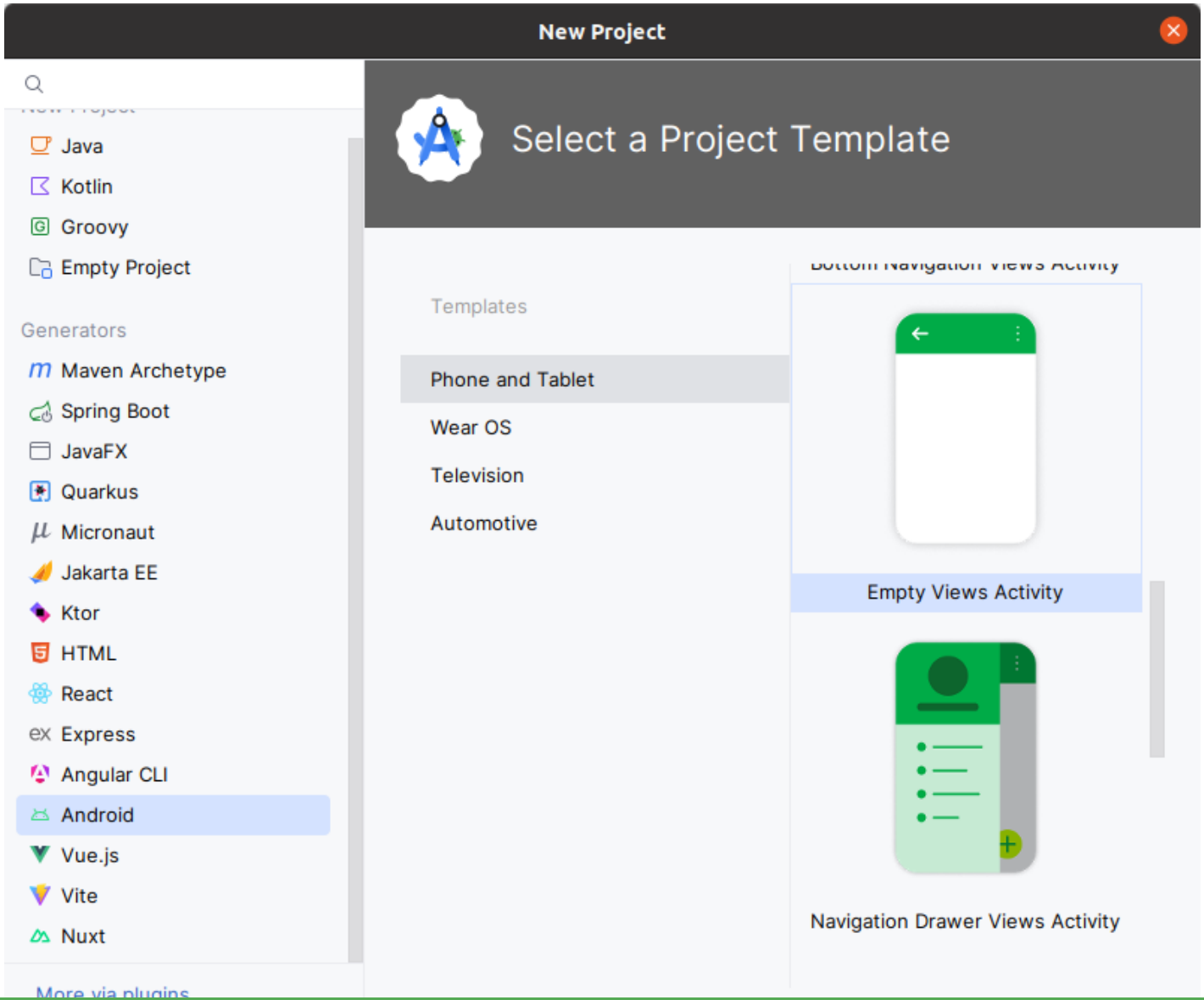


Add Android Plugin

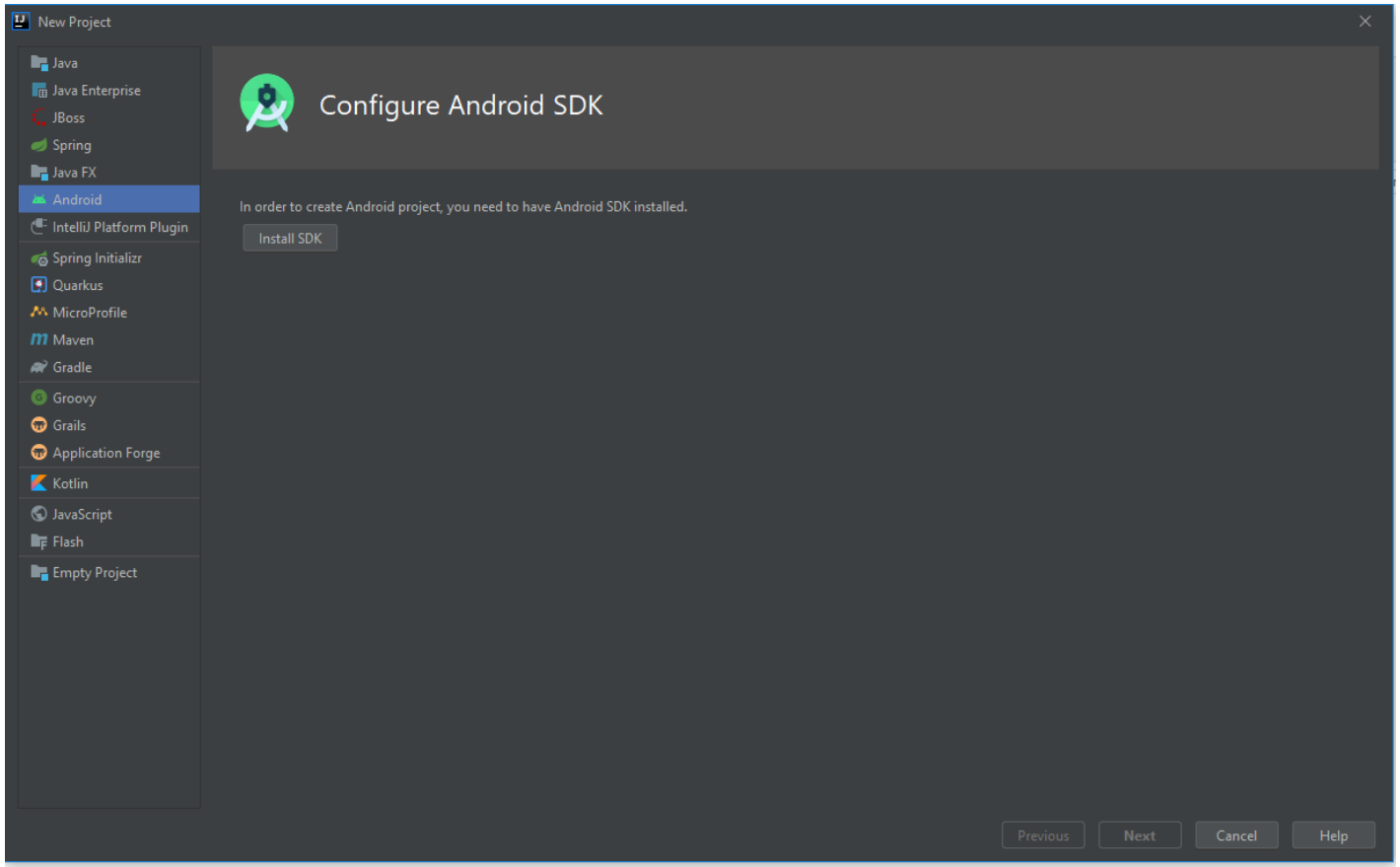
- The IntelliJ IDEA shows the installed plugins.
- Check `Android` and click `Install` and after installation click `Restart IDE`



Create a Project (I)

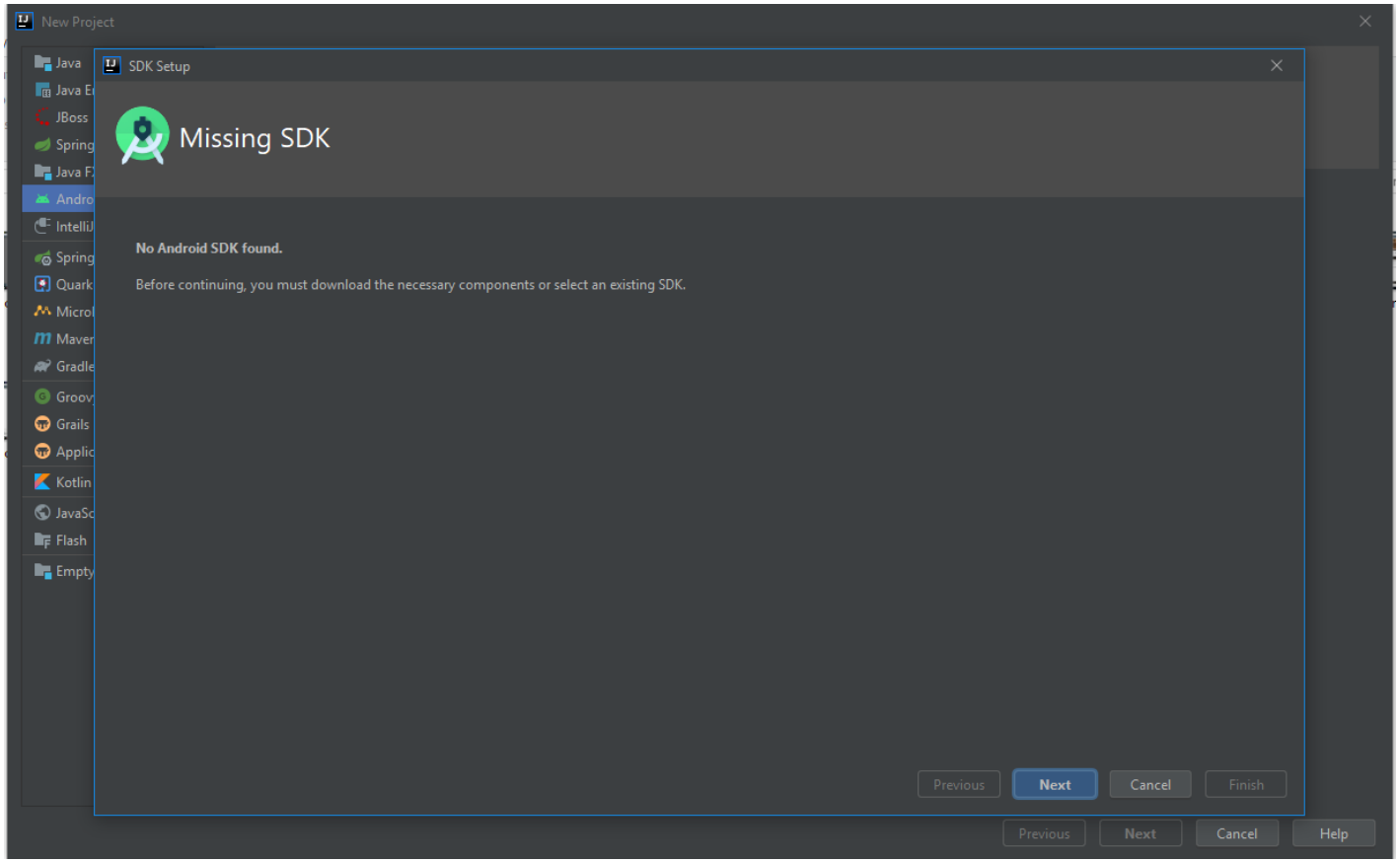


Install Android SDK (I)



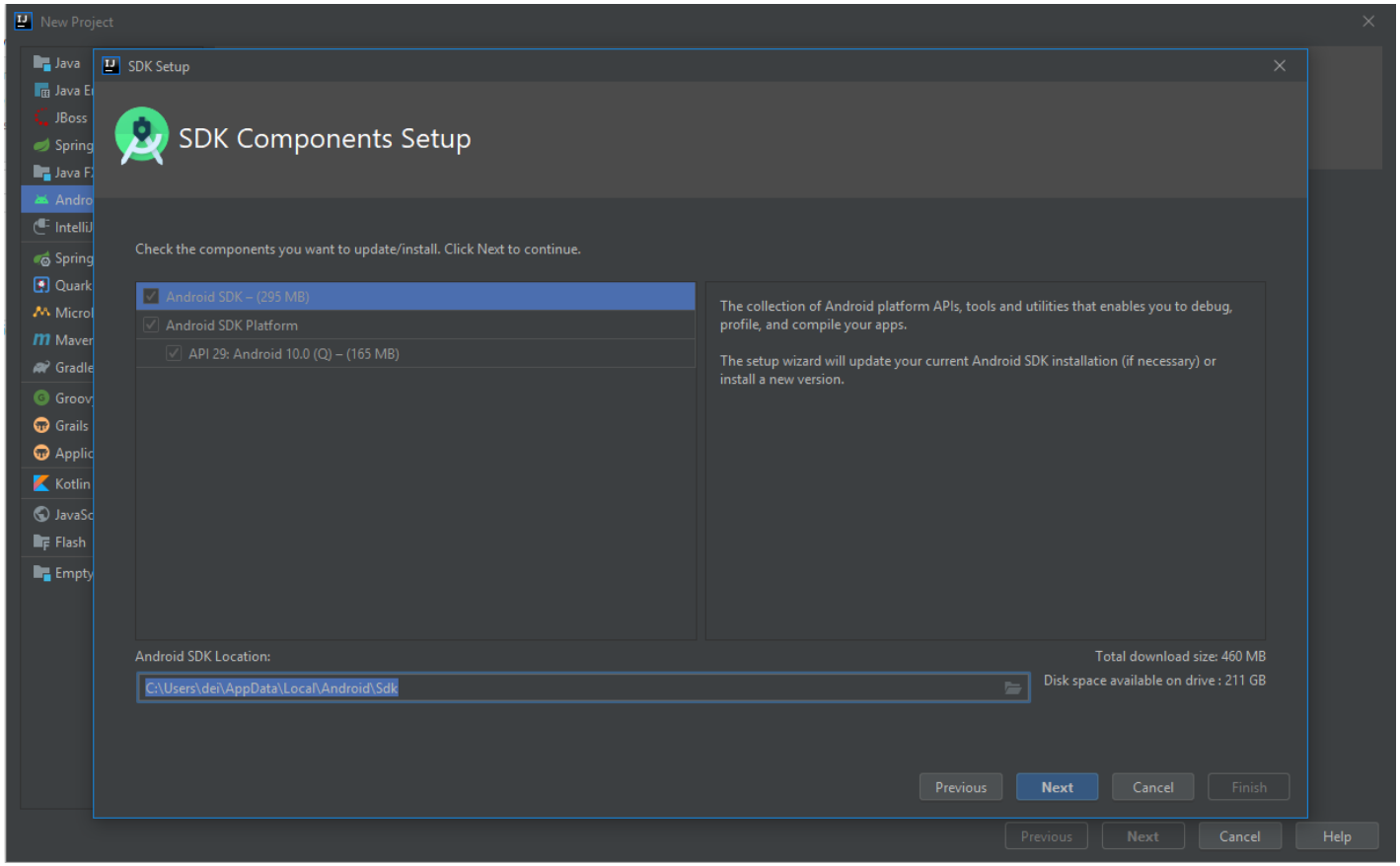
- Select `Android`
- Click on `Install SDK`

Install Android SDK (II)



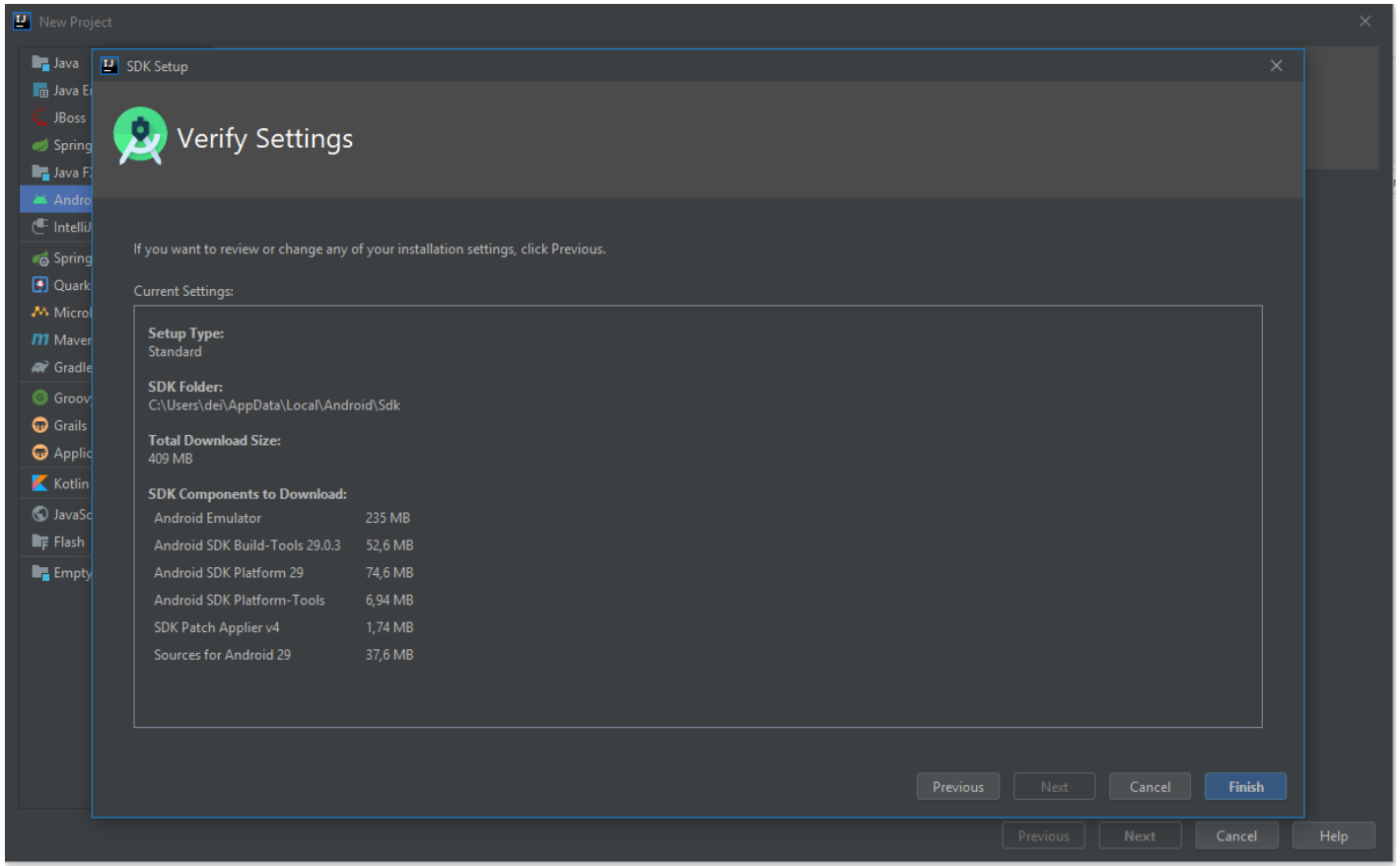
- Click on `Next`

Install Android SDK (III)



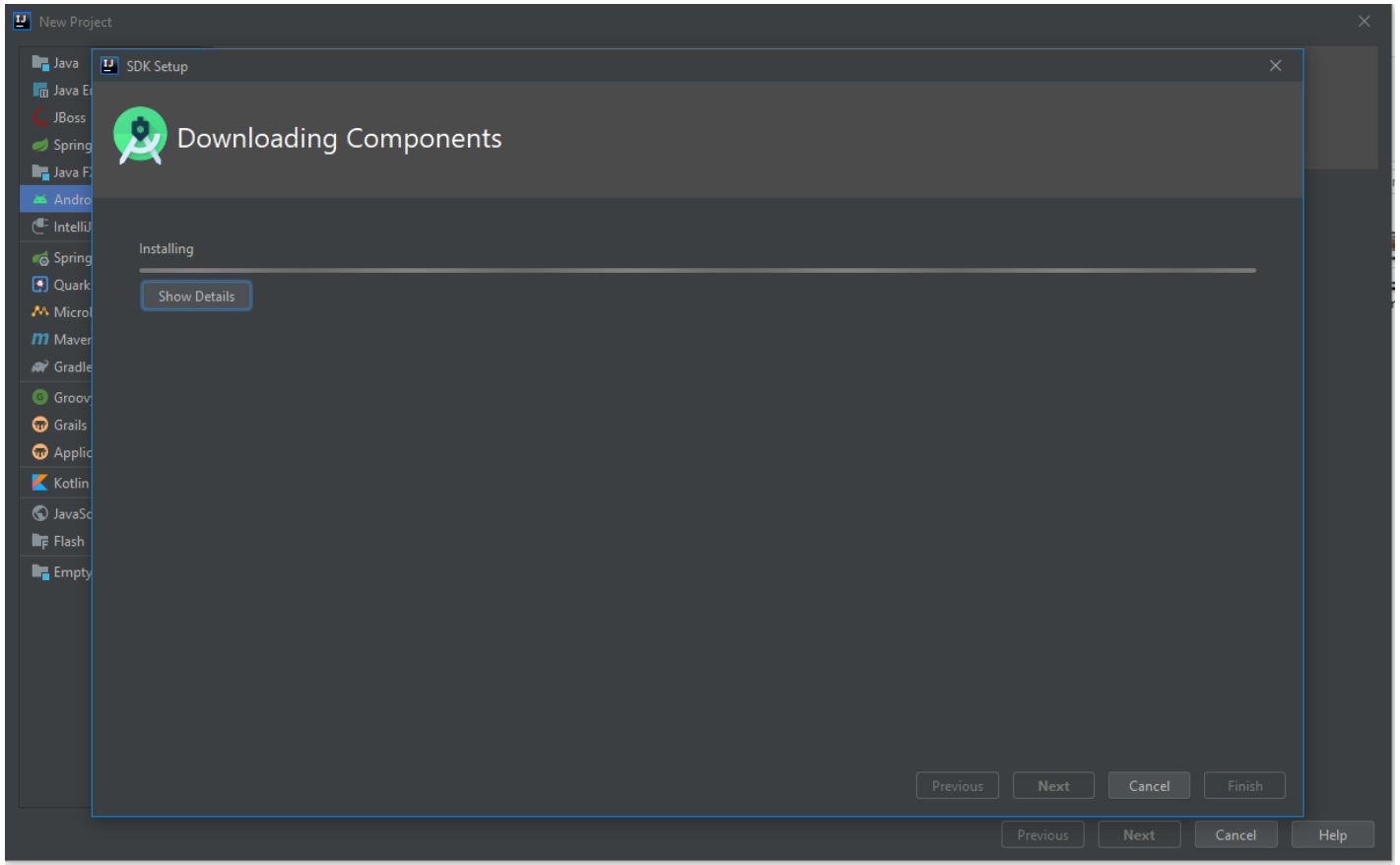
- Click on `Next`

Install Android SDK (IV)

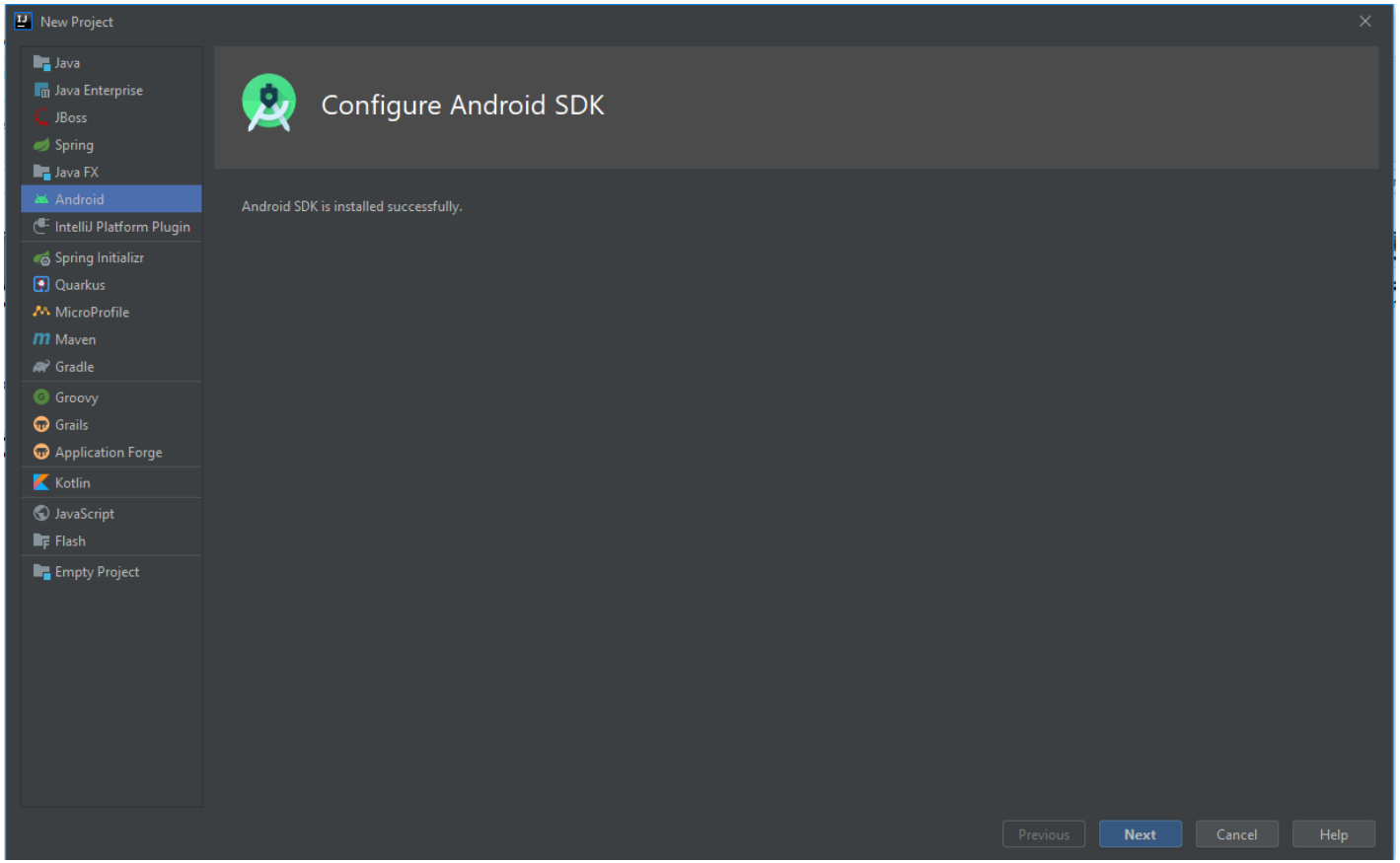


- Click on `Finish`

Install Android SDK (V)

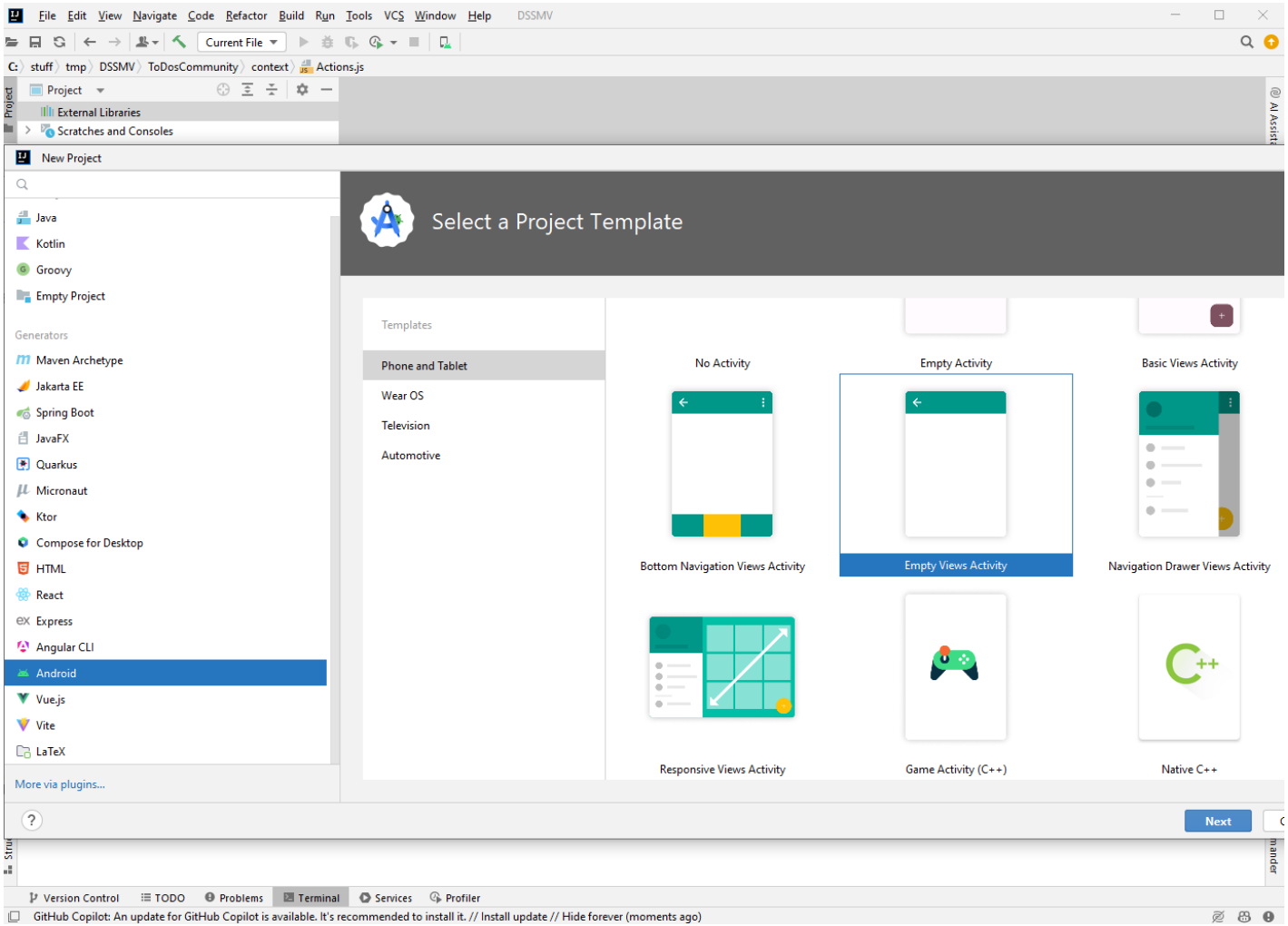


Install Android SDK (VI)



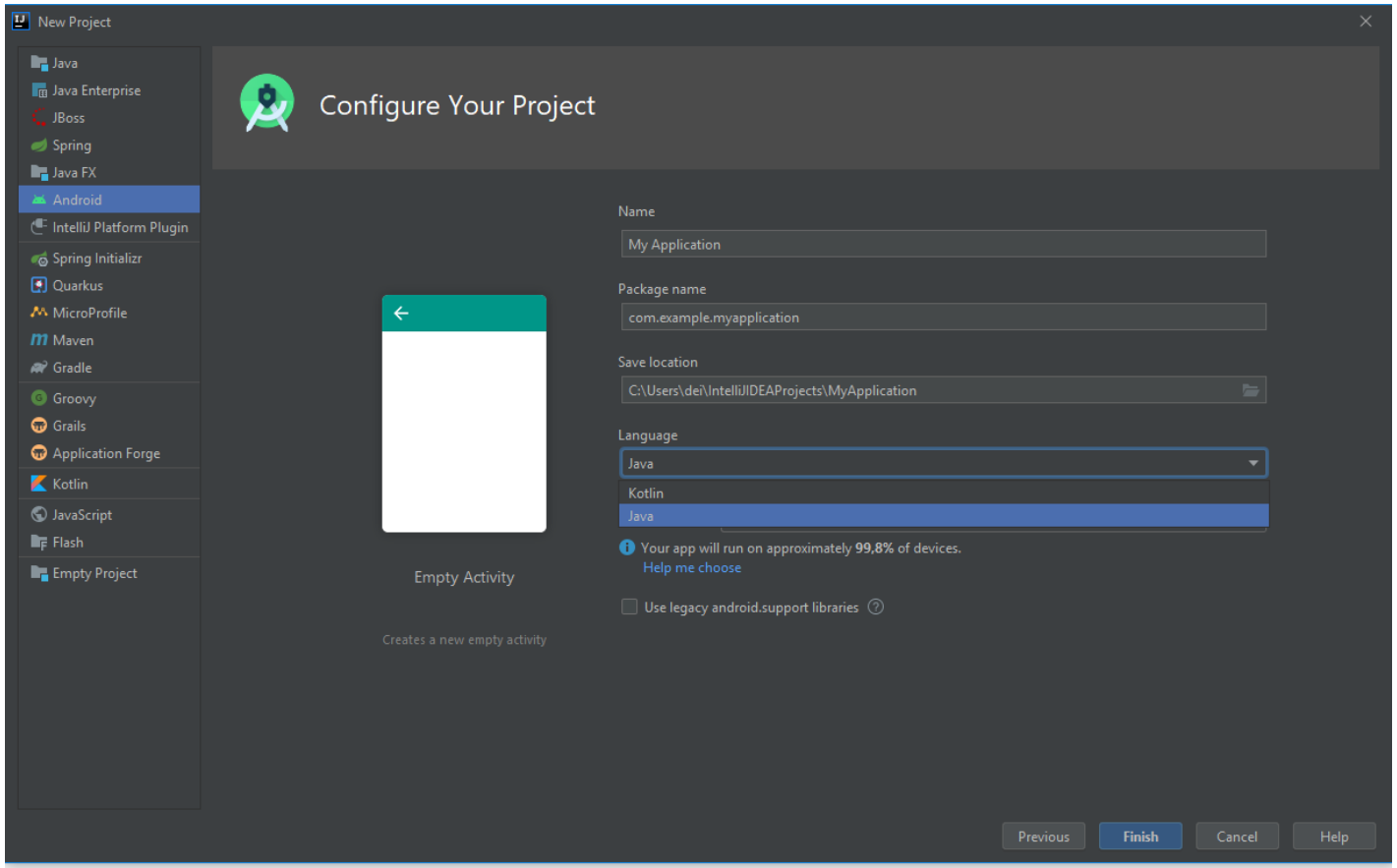
- Click on `Next`

Create a Project (II)



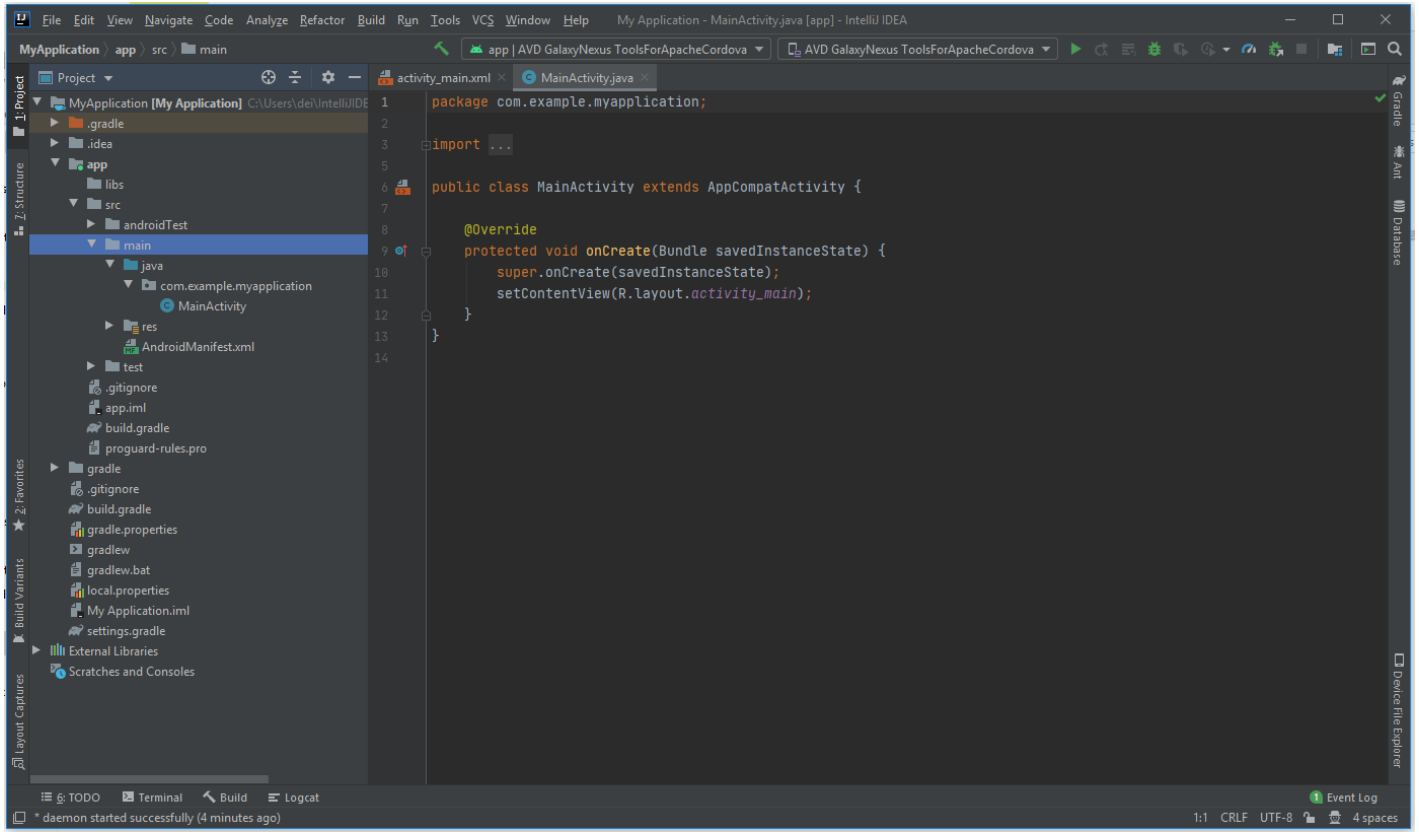
- Select `Empty Views Activity`

Create a Project (III)

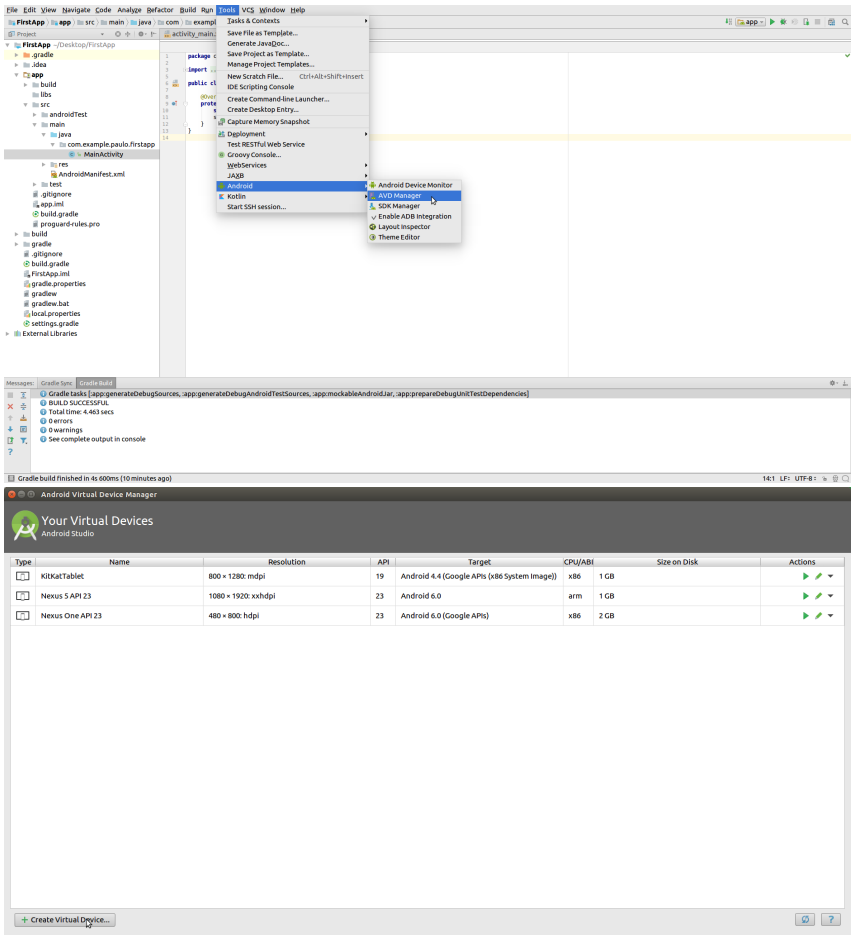


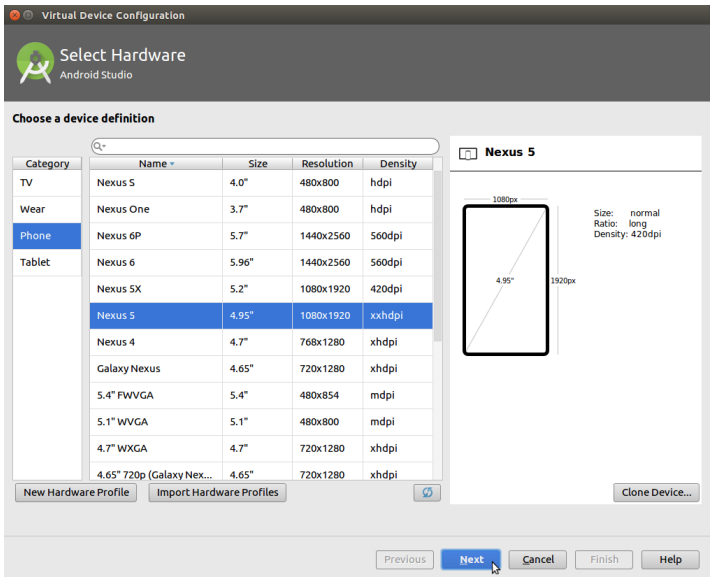
- Select Language `Java`

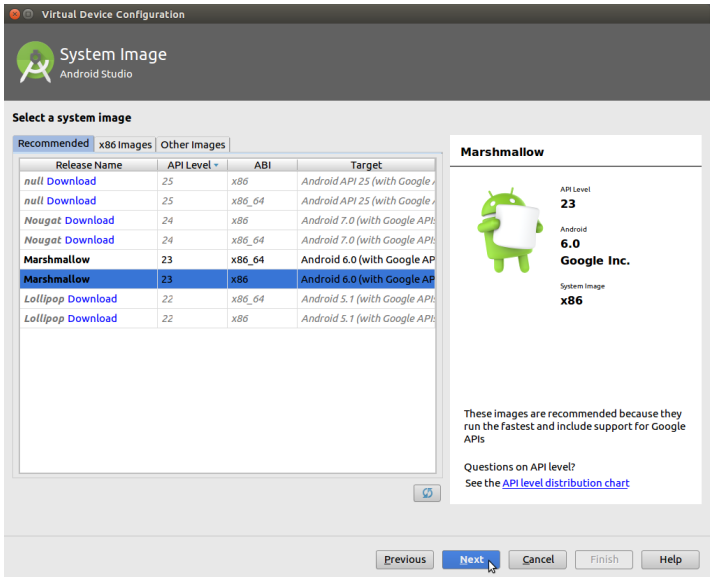
Create a Project (IV)



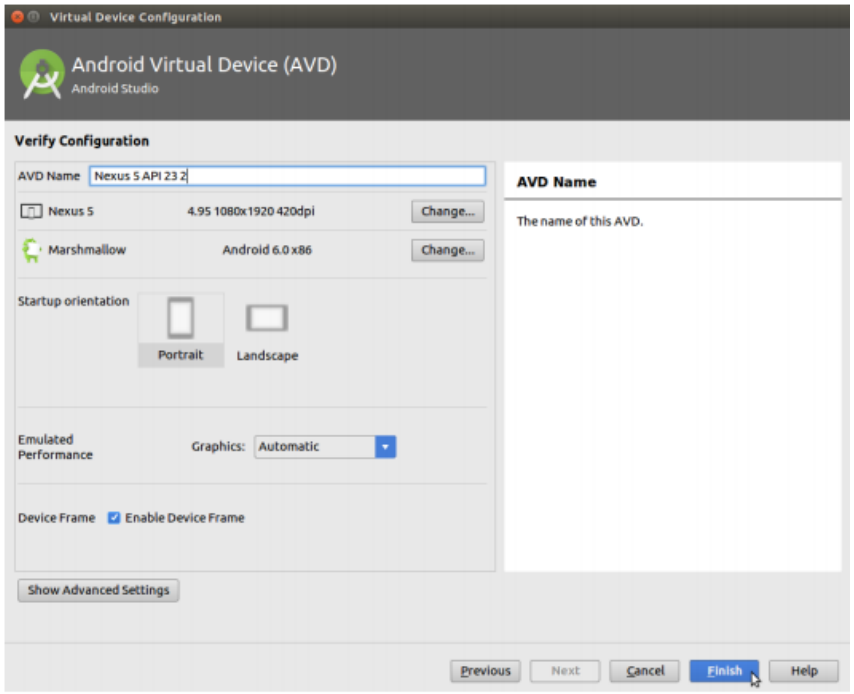
Create an Android Virtual Device (AVD) (I)

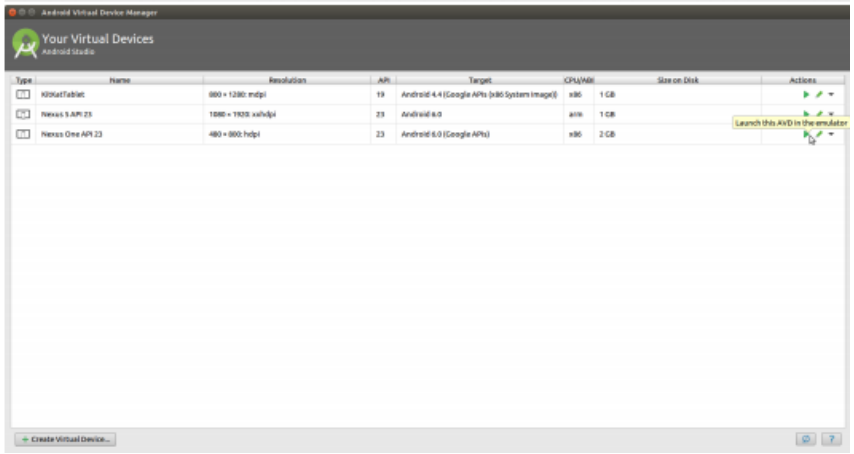


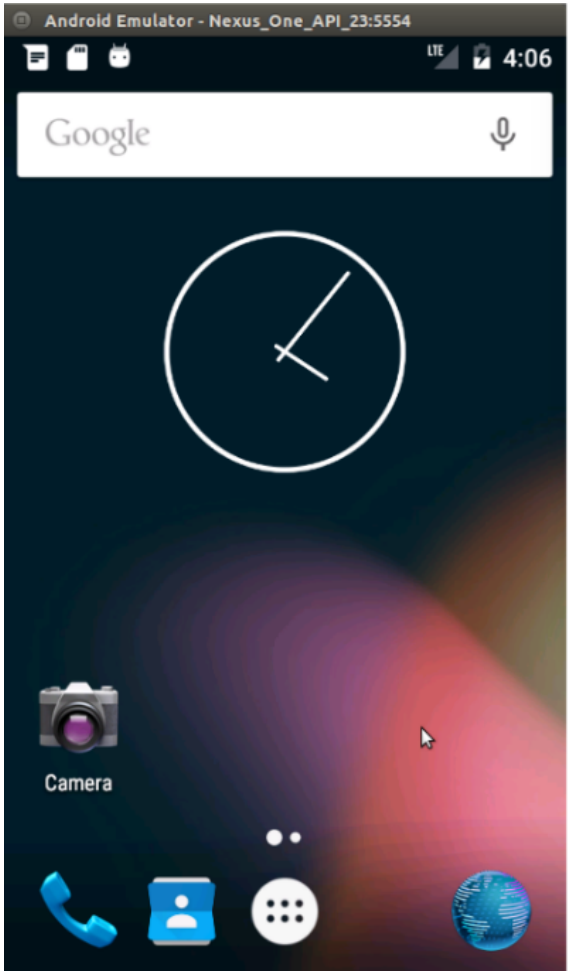




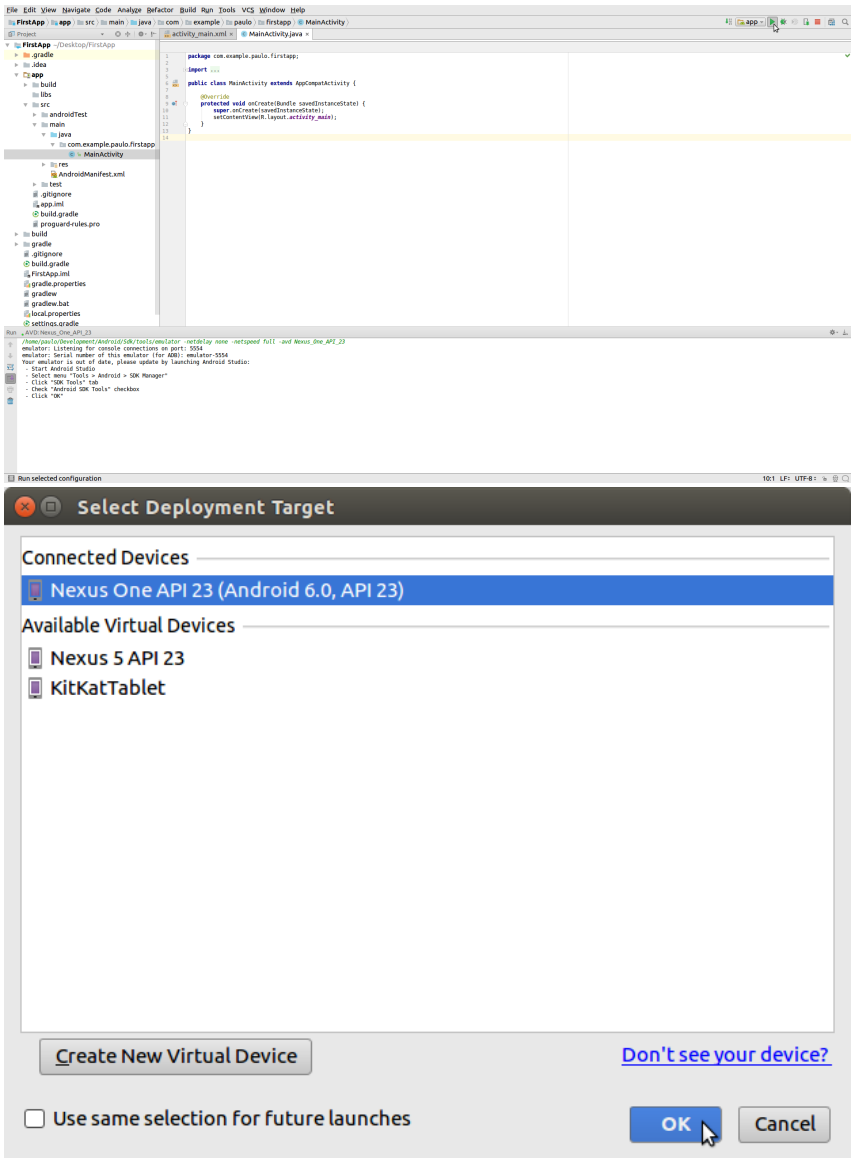
Create an Android Virtual Device (AVD) (II)

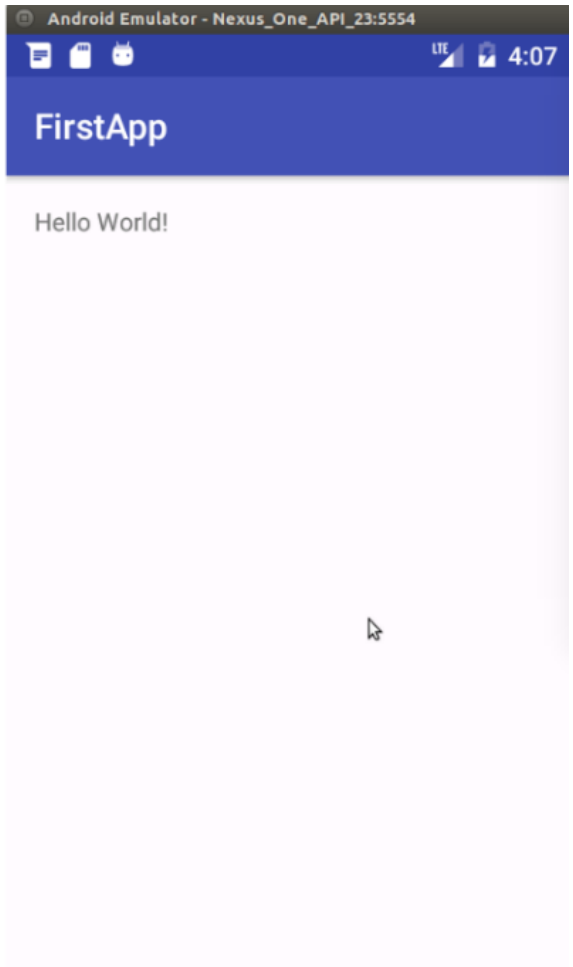




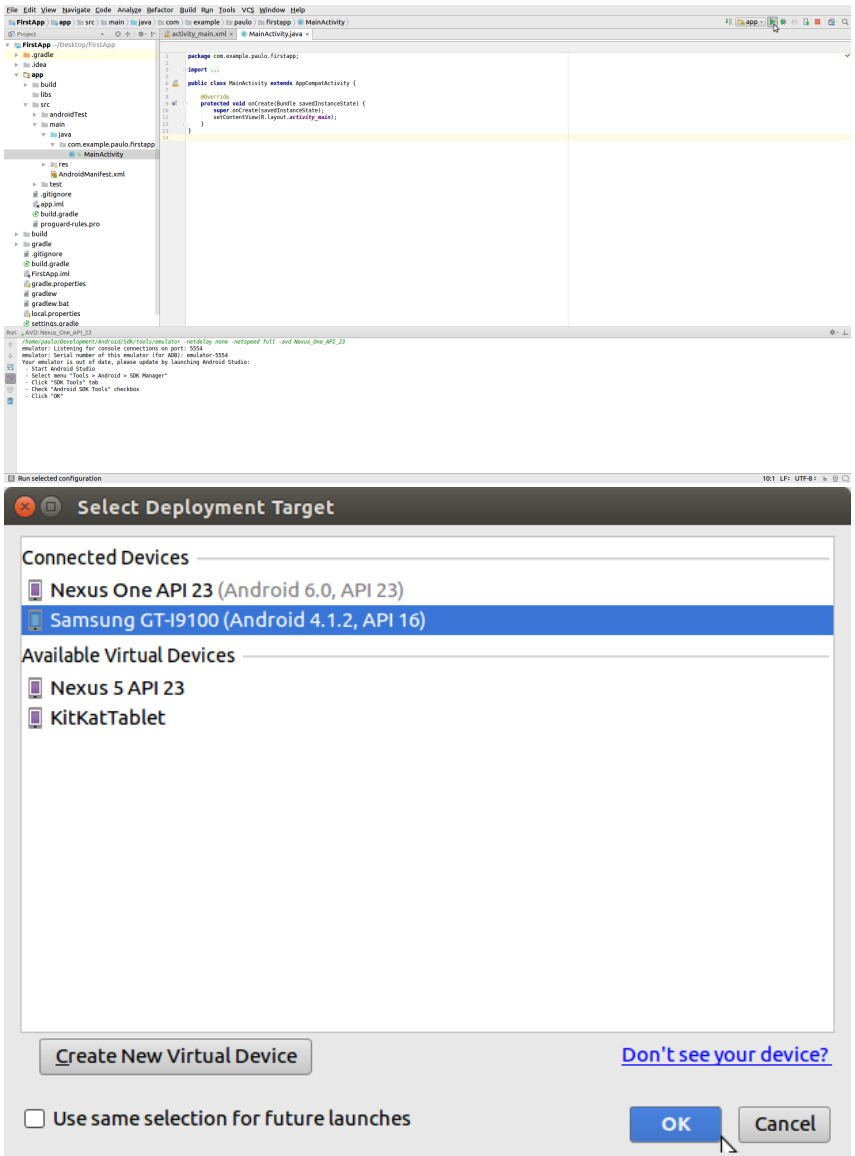


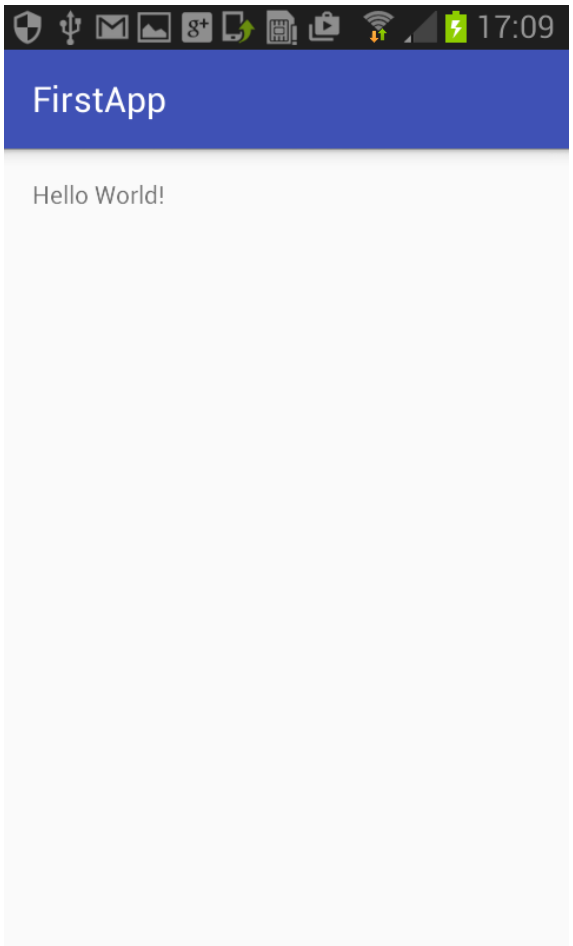
Running the FirstApp on AVD





Running the FirstApp on Real Device





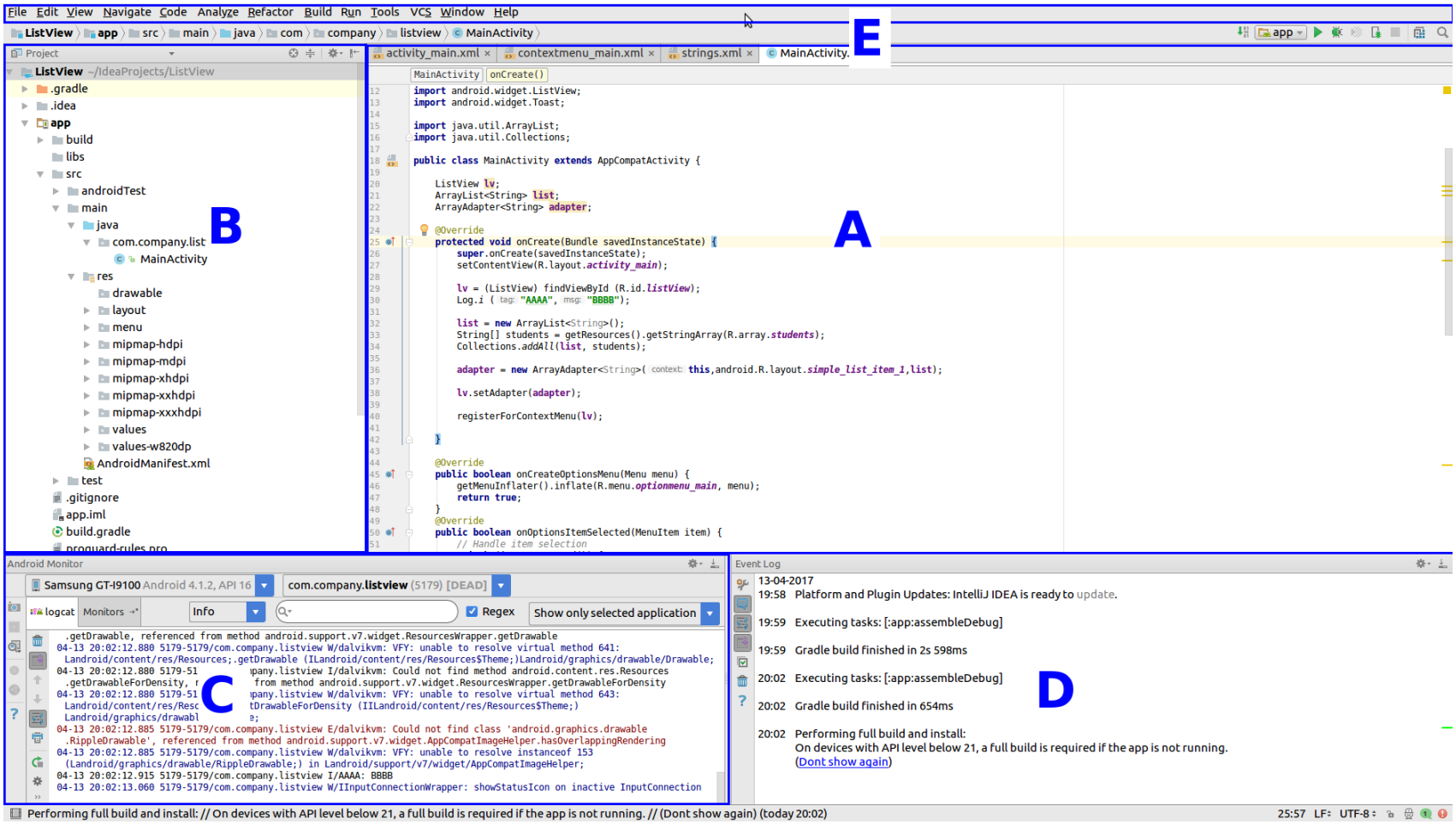
IntelliJ IDEA

Overview

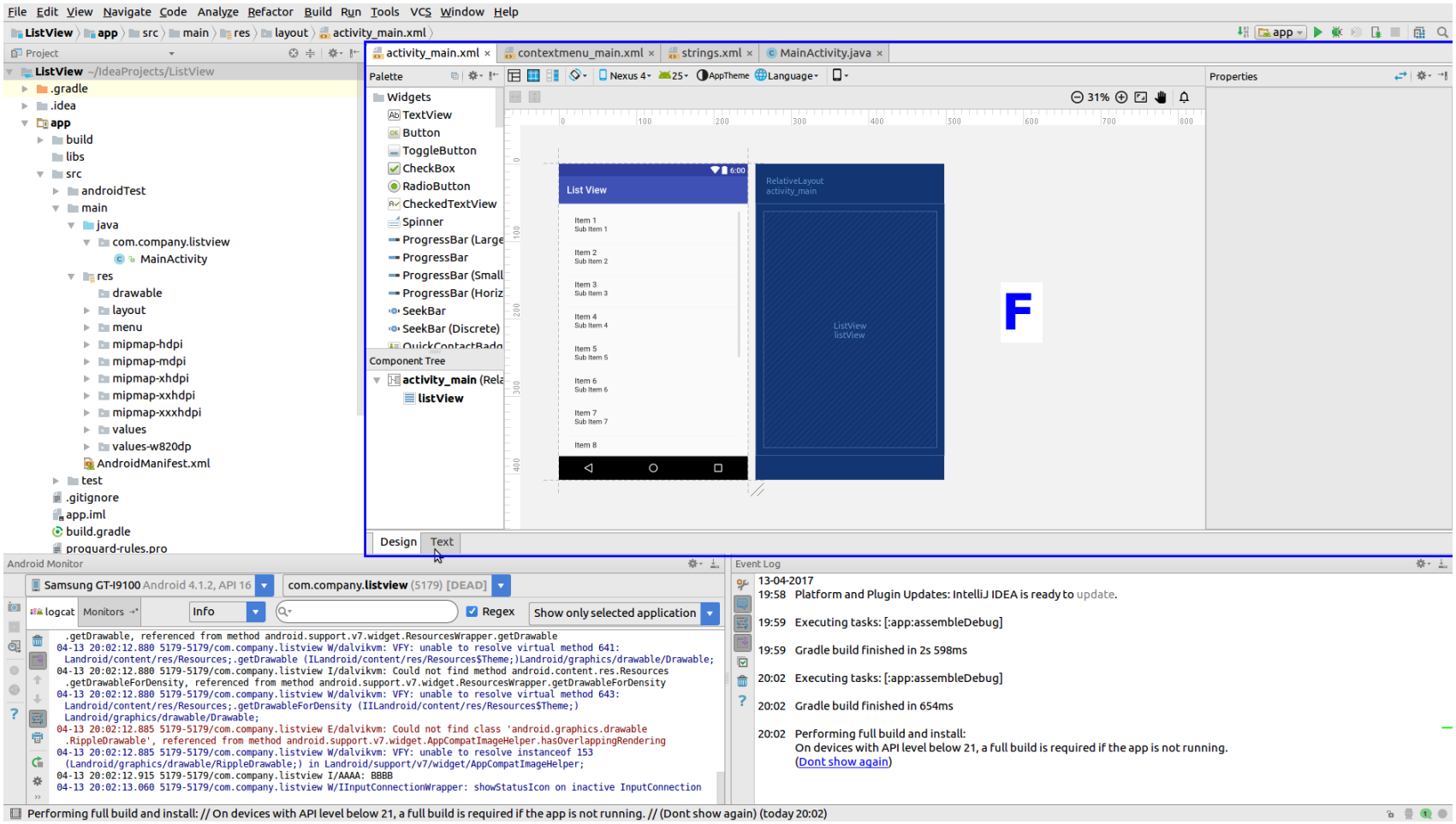
- Android Software Development Kit (SDK) is required for developing Android apps.
  - It includes everything required for developing Android apps:
    - Android SDK Tools
    - Android Platform-tools
    - Android platform versions
    - Android system images (for the emulator)
- Android Studio [1] is the official IDE.
- IntelliJ IDEA could also be used for developing Android Apps

[1] developer.android.com/sdk/index.html

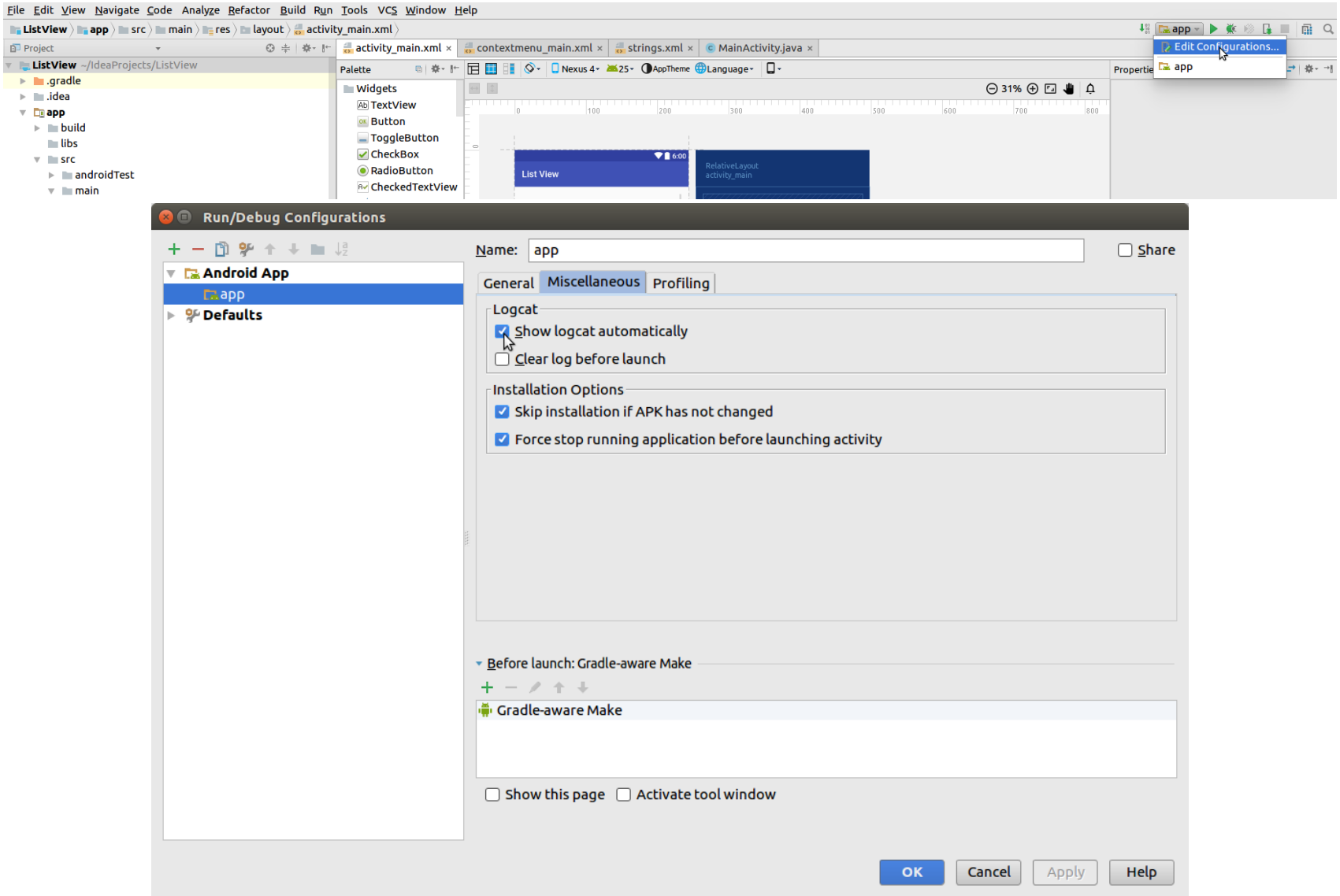
IntelliJ IDEA: General view (I)



IntelliJ IDEA: General view (II)



IntelliJ IDEA: Logcat



Basics

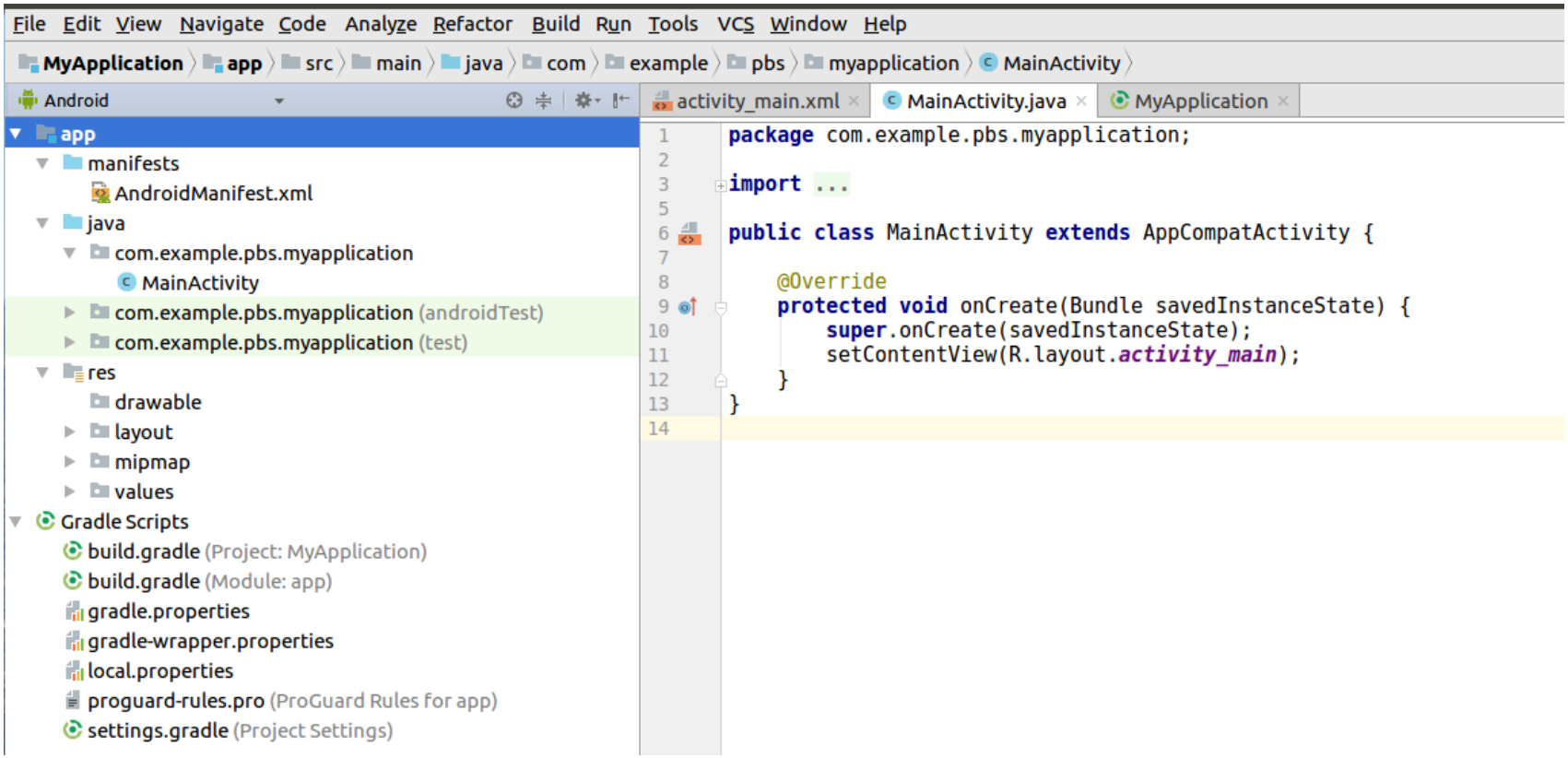
Android Application

- An Android application (**app**) is a single installable unit which can be started and used independently.
- An Android app consists of configuration files, Java source and resource files
- Android Package (APK) is the package file format used by the Android operating system for distribution and installation of mobile apps.

Android view (I)

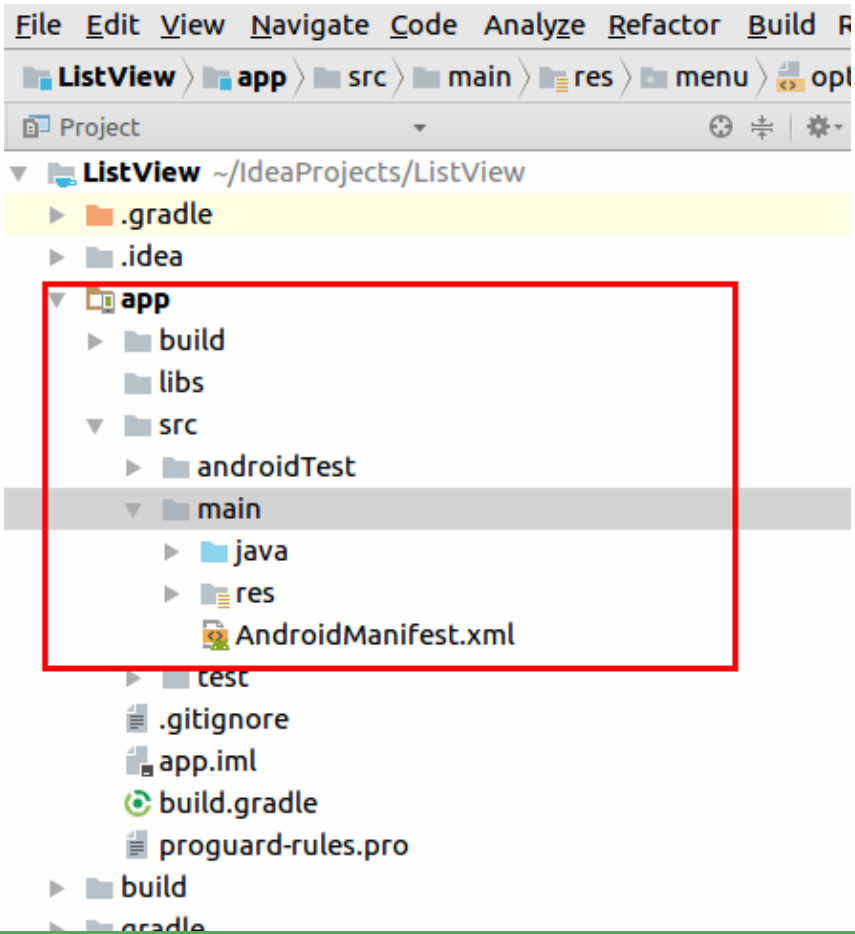
- This view shows a flattened version of your project's structure that provides quick access to the key source files of Android projects and helps you work with the Gradle-based build system.
- Within each Android app module, files are shown in the following groups:
  - `AndroidManifest.xml` file.
  - `java`
    - Contains the Java source code files, separated by package names.
  - `res`
    - Contains all non-code resources, such as XML layouts, UI strings, and bitmap images, divided into corresponding sub-directories. For more information about all possible resource types, see Providing Resources.
  - `GradleScripts`
    - Gradle build and property files.

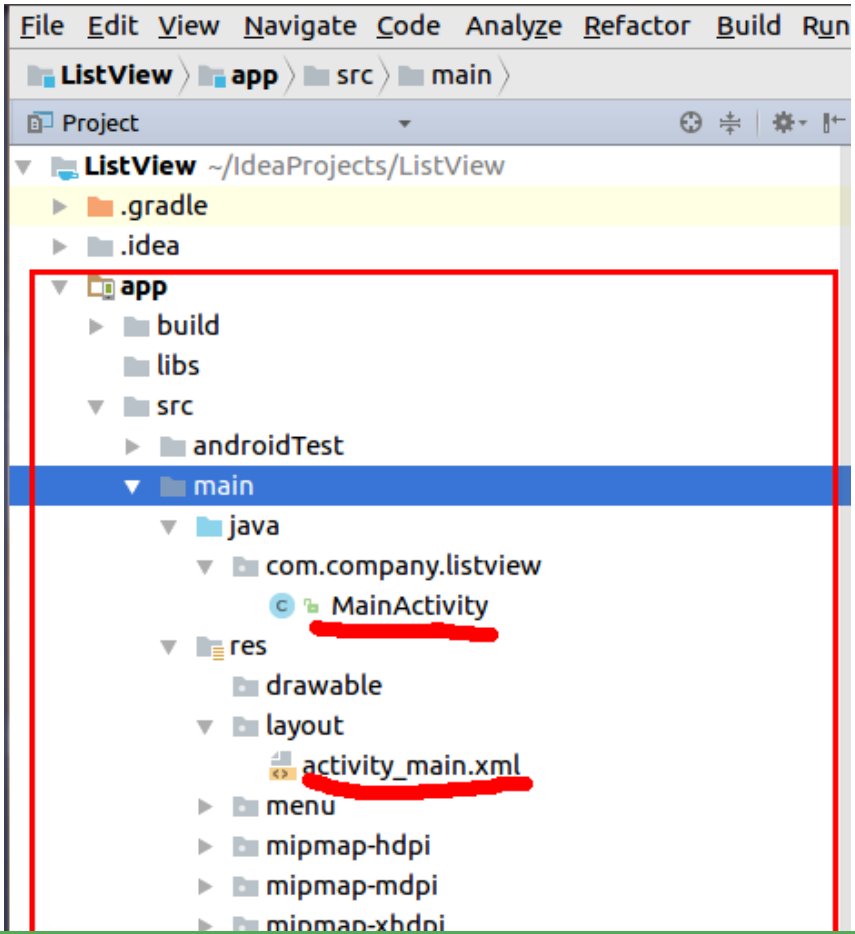
Android view (II)



Project view

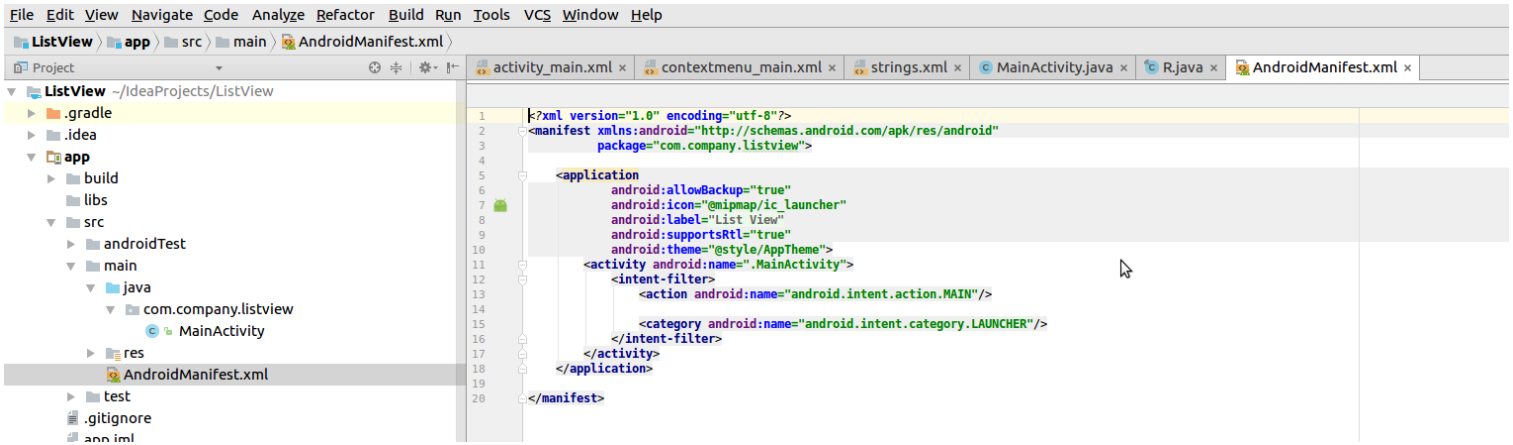
- In this view, all the project items along with their dependencies (SDKs and libraries) are shown. The emphasis is on the directory structure (though the packages are also shown).





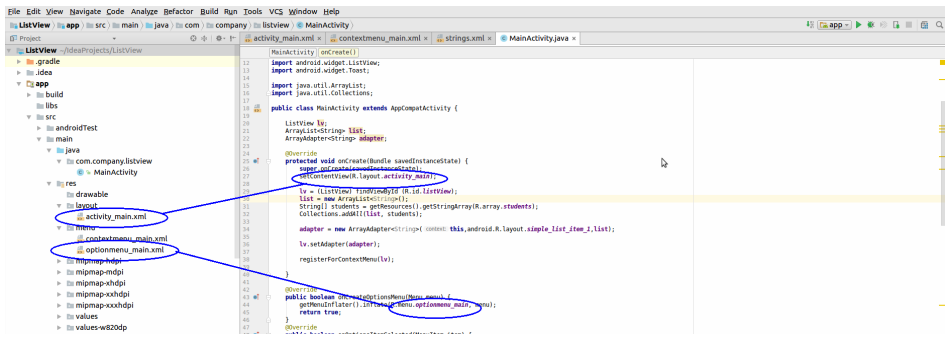
Android Manifest XML

- Every app **must have** a `AndroidManifest.xml` file in its root directory.
- The manifest file **presents essential information about app** to the Android system:
  - Java package name for the application: the package name serves as a unique identifier for the application.
  - The components of the application: the activities, services, broadcast receivers, and content providers.
  - Permissions the application.



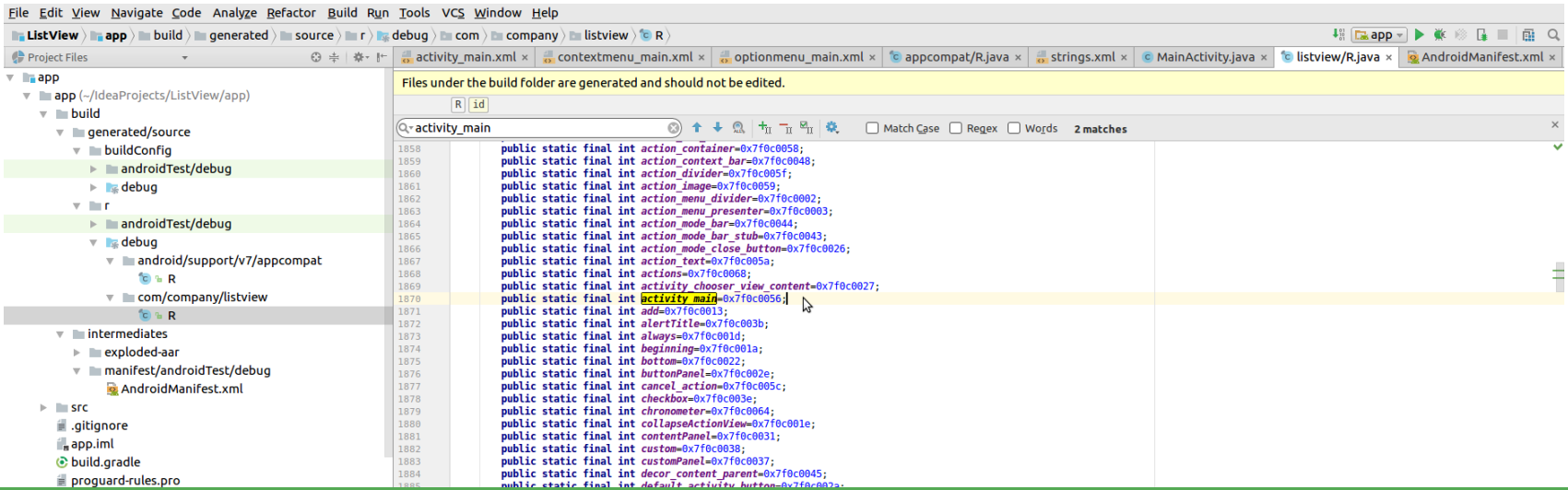
Model-View-Controller (MVC)

- Programming rules establishes a separation between User Interface (UI) and java code:
  - Models:
    - Shared Preferences
    - SQLite Databases
    - Network Connection
    - Internal and External Storage
    - Syncing to the Cloud
  - Views
    - XML files
  - Controllers
    - Java files



Class `R`

- **All resource Identifiers** (IDs) are defined in your project's `R` class, which the `aapt` tool automatically generates.
- `R` class contains resource IDs for all the resources in your `res/` directory.
  - For each type of resource, there is an `R` subclass (for example, `R.layout` for all layout resources), and for each resource of that type, there is a static integer (for example, `R.layout.activity_main`).
    - This integer is the resource ID that you can use to retrieve your resource.



Dimension Units

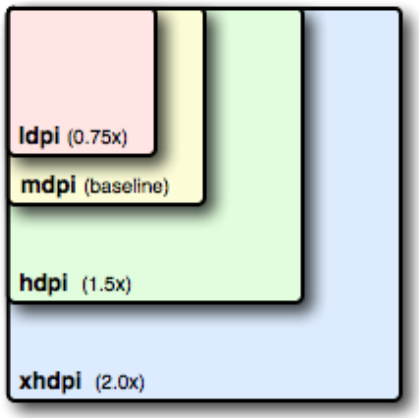
- In order to express a form of distance or length, there are several units for dimensions.
  - These can be used on elements to set widths, heights, margins, padding, and more:
    - `px`: An actual pixel on the screen. This is a density dependent unit, and the physical size of a single "px" varies depending on screen density.
    - `in`: A physical inch on the screen.
    - `mm`: A physical millimeter on the screen.
    - `pt`: A point, a common font size unit, on the screen.
    - `dp`: This is a density independent unit, however the physical size of a single "dp" is only approximately the same on every screen density. There are approximately 160 "dp" in an inch.
    - `sp`: A scale independent pixel, specially designated for text sizes.

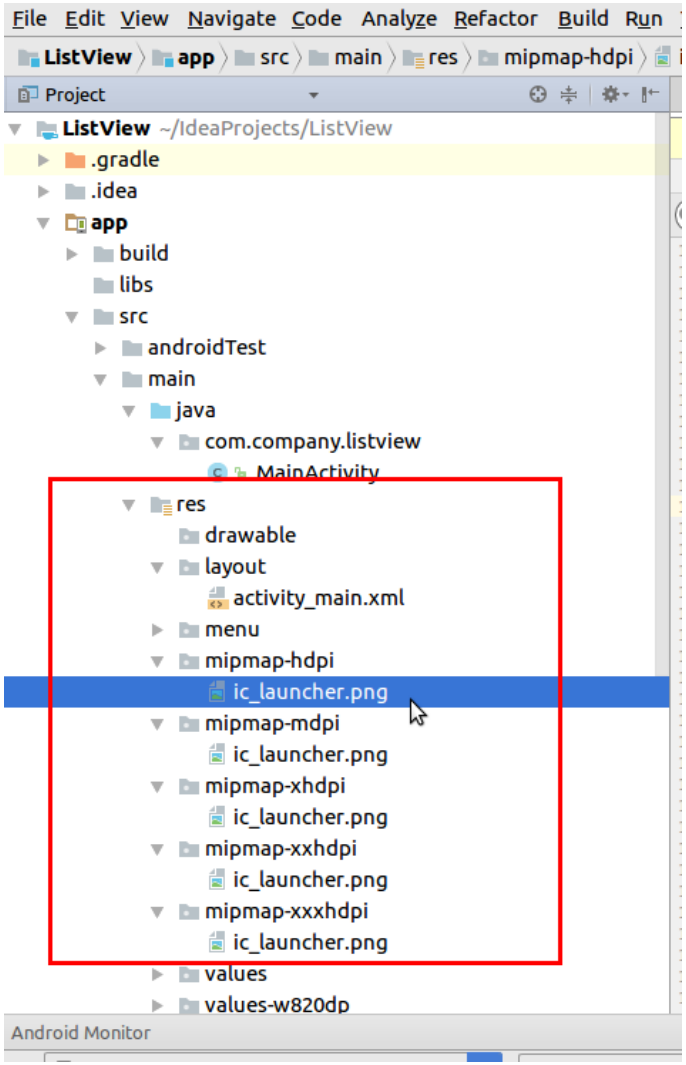
Preferrable

It must be used the **Independent** units

Images

- Since Android runs in devices with a wide variety of screen densities, you should always provide your bitmap resources tailored to each of the generalized density buckets: low, medium, high and extra-high density.
  - This will help you achieve good graphical quality and performance on all screen densities.





`Context`

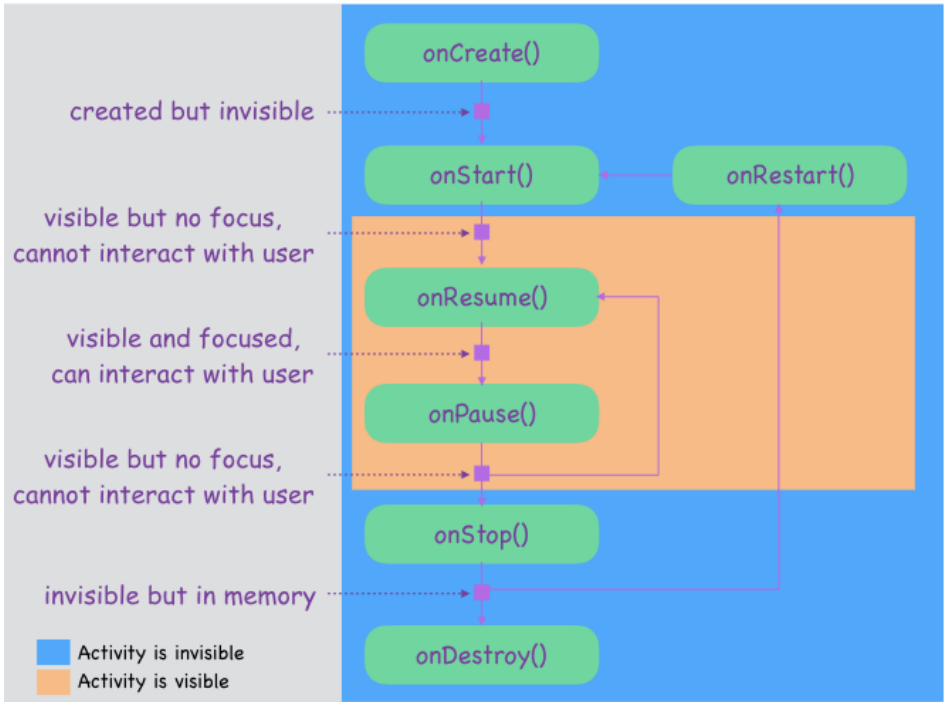
- Instances of the class `android.content.Context` provide the **connection to the Android system and actual device**.
- It gives access to the system and application resources and services.
  - For example, you can check the size of the current device display via it.
- Activities and services extend the `Context` class. Therefore, they can be directly used to access the `Context`.
  - `getApplicationContext()`
  - `getContext()`
  - `getBaseContext()`
  - `this` (when in the activity class)

Activity

- An `Activity` represents a single screen with a user interface.
- Android system initiates its program with in an `Activity` starting with a call on `onCreate()` callback method.

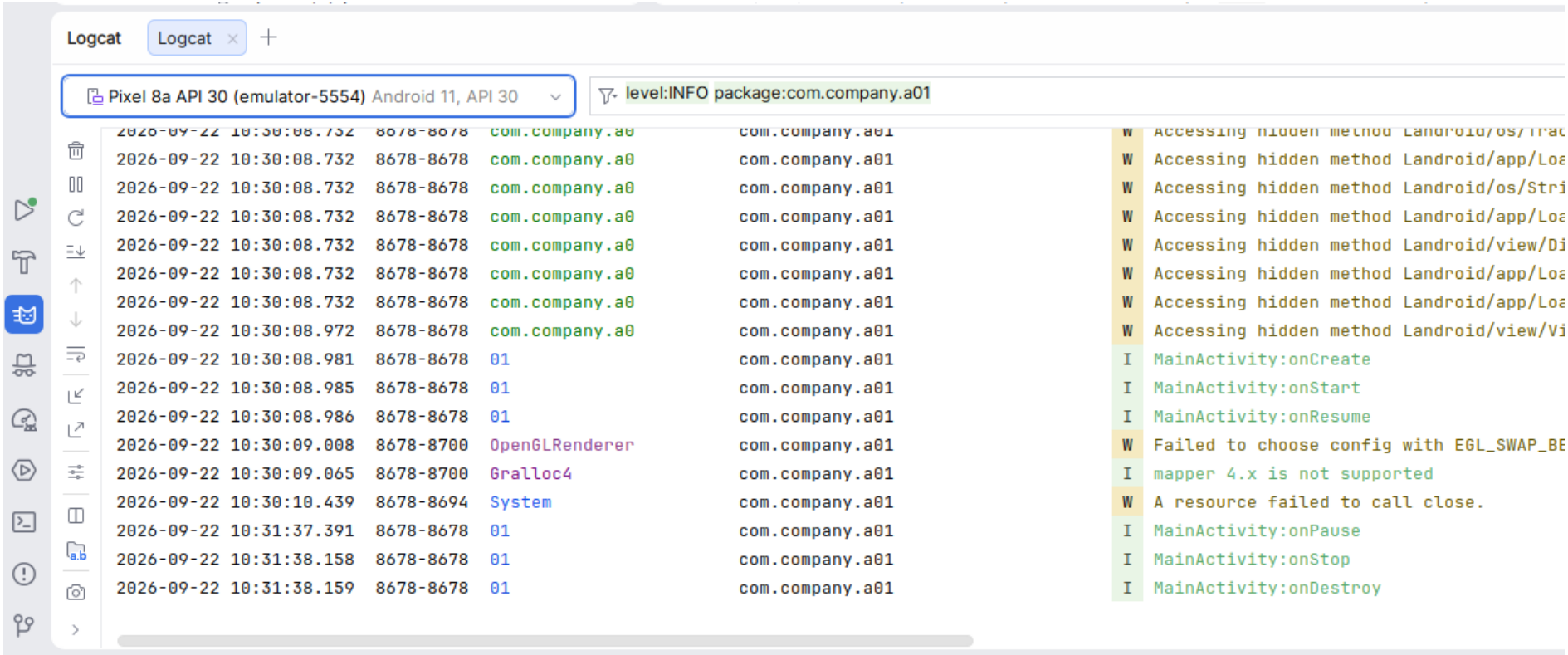
```
public class MainActivity extends
      Activity {
   @Override
   public void onCreate(Bundle
         savedInstanceState) {
      super.onCreate(
            savedInstanceState);
      }
}
```

- There is a sequence of callback methods that start up an activity and a sequence of callback methods that tear down an activity.



Check: `TP03_01`

`Activity` Lifecyle



Check: `TP03_01`

AppCompatActivity

- `AppCompatActivity` provides backward compatibility.

```
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

  - `AppCompatActivity` is a subclass of `Activity`
- It requires the `appcompat` library (`build.gradle`)

```
...
dependencies {
    implementation(libs.appcompat)
    ...
}
```

Linking a Layout to an Activity

- An `Activity` class loads all the UI component using the XML file available in `res/layout` folder of the project by invoking `setContentView` method.
  - `.../MainActivity.java`

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

  - `res/layout/activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
      ...
       android:layout_width="match_parent"
       android:layout_height="match_parent">
    <ListView
          android:layout_width="match_parent"
          android:layout_height="wrap_content"/>
</RelativeLayout>
```

Mapping a UI component on Java Variable

- `findViewById(int)`
  - Retrieves the UI component (loaded by `setContentView`).
  - Finds a view that was identified by the `id` attribute from the XML.

```
public class MainActivity extends AppCompatActivity {
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      ListView lv = (ListView) findViewById (R.id.listView);
   }
}
```

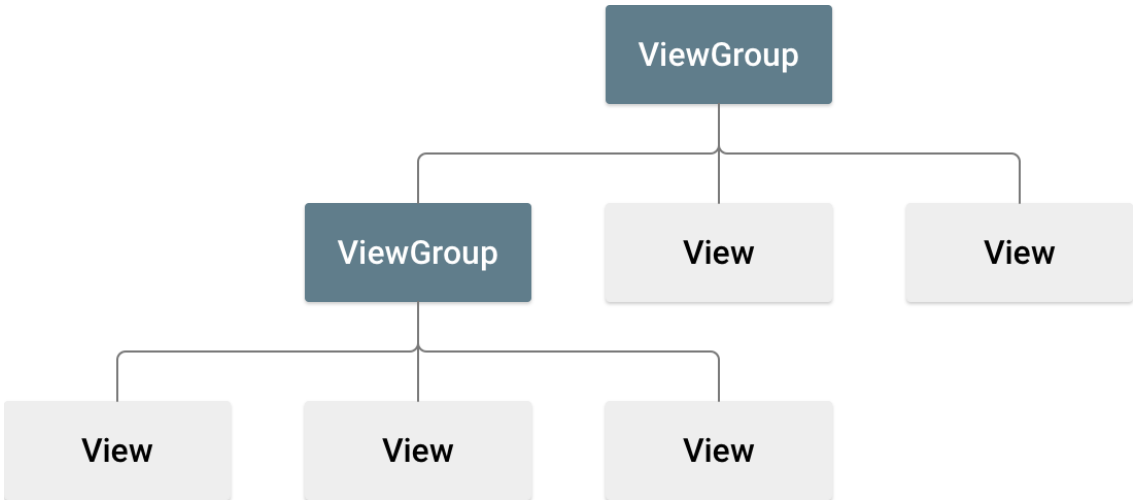
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
      ...">
   <ListView
         android:layout_width="match_parent"
         android:layout_height="wrap_content"
      @@ android:id="@+id/listView" @@/>
</RelativeLayout>
```

Check: `TP03_02`

Graphical Interface

Basics on Graphical Interface

- The user interface for an Android app is built using a hierarchy of layouts (`ViewGroup` objects) and widgets (`View` objects).
- **Layouts are invisible containers** that control how its child views are positioned on the screen.
- **Widgets are UI components** such as buttons and text boxes.
- Android provides an XML vocabulary for ViewGroup and View classes, so most of your UI is defined in XML files.

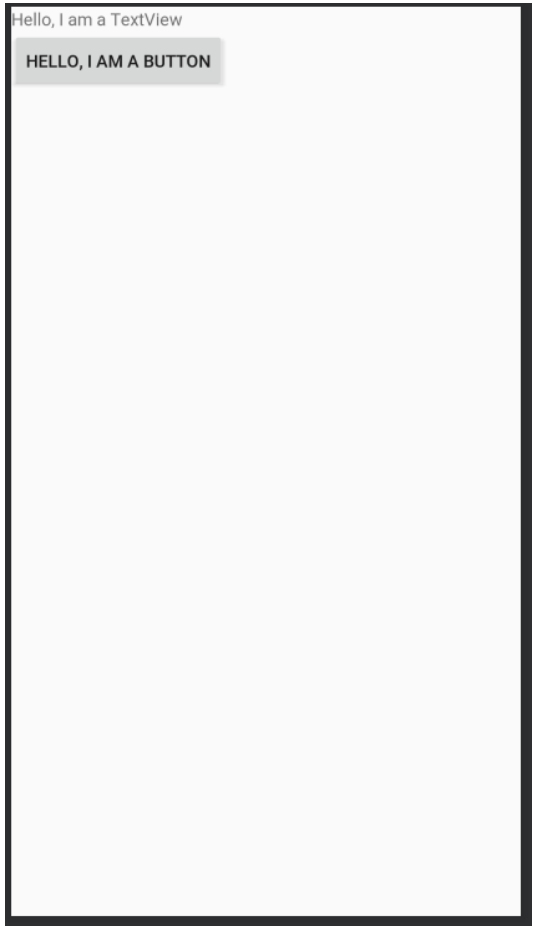


Layouts

- A layout **defines the visual structure** for a user interface, such as the UI for an `Activity`.
- You can declare a layout in two ways:
  - Declare UI elements in XML (**Recommended**).
    - Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.
  - Instantiate layout elements at runtime.
    - Your application can create View and ViewGroup objects (and manipulate their properties) programmatically.

Layouts: XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
          xmlns:android="http://schemas.android.
               com/apk/res/android"
          android:layout_width="match_parent"
          android:layout_height="match_parent"
          android:orientation="vertical" >
   <TextView android:id="@+id/text"
          android:layout_width="wrap_content"
          android:layout_height="wrap_content"
          android:text="Hello, I am a TextView" />
   <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```



Layouts: Atributes

- Every `View` and `ViewGroup` object supports their own variety of XML attributes.
- Some attributes are specific to a `View` object (for example, `TextView` supports the `textSize` attribute).
- Some are common to all `View` objects, because they are inherited from the root View class (like the `id` attribute).
- Other attributes are considered "layout parameters," which are attributes that describe certain layout orientations of the `View` object, as defined by that object's parent `ViewGroup` object.

Layouts: Atribute ID

- Any `View` object may have **an integer ID** associated with it, to uniquely identify the View within the tree.
- When the app is compiled, this ID is referenced as an integer, but the ID is typically assigned in the layout XML file as a string, in the `id` attribute.

```
android:id="@+id/my_button"
```

  - The at-symbol (@) at the beginning of the string indicates that the XML parser should parse and expand the rest of the ID string and identify it as an ID resource.
  - The plus-symbol (+) means that this is a new resource name that must be created and added to our resources (in the `R.java` file).

Layouts: Parameters

- XML layout attributes named `layout_something` define layout parameters for the View that are appropriate for the `ViewGroup` in which it resides.
- All views include a width and height (`layout_width` and `layout_height`), and each view is required to define them.
  - `wrap_content` tells your view to size itself to the dimensions required by its content.
  - `match_parent` tells your view to become as big as its parent view group will allow.
- In general, **specifying a layout width and height using absolute units such as pixels is not recommended**.
  - Instead, using relative measurements such as density-independent pixel units (`dp`), `wrap_content`, or `match_parent`, is a better approach.

Layouts: Common

- `LinearLayout`
  - A layout that organizes its children into a single horizontal or vertical row. It creates a scrollbar if the length of the window exceeds the length of the screen.

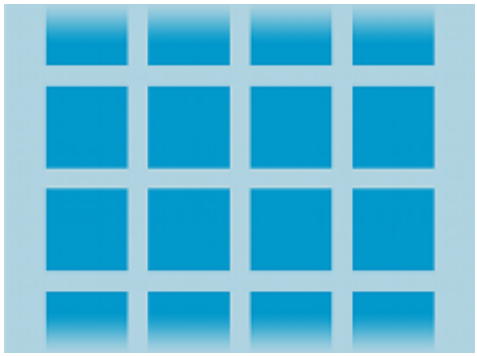


- `RelativeLayout`
  - Enables you to specify the location of child objects relative to each other (child A to the left of child B) or to the parent (aligned to the top of the parent).



Layouts: backed by an Adapter

- When the content for your layout **is dynamic** or not pre-determined, you can use a layout that subclasses `AdapterView` to populate the layout with views at runtime.
  - A subclass of the `AdapterView` class uses an `Adapter` to bind data to its layout.
  - The Adapter behaves as a middleman between the data source and the `AdapterView` layout
    - the `Adapter` retrieves the data (from a source such as an array or a database query) and converts each entry into a view that can be added into the `AdapterView` layout.

- `List View` 
- `Grid View` 

Input Controls (I)

- Input controls are the interactive components in your app's user interface.
  - Android provides a wide variety of controls you can use in your UI, such as buttons, text fields, seek bars, checkboxes, zoom buttons, toggle buttons, and many more.
- Adding an input control to your UI is as simple as adding an XML element to your XML layout.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="fill_parent"
   android:layout_height="fill_parent"
   android:orientation="horizontal">
   <EditText android:id="@+id/edit_message"
      android:layout_weight="1"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:hint="@string/edit_message" />
   <Button android:id="@+id/button_send"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/button_send"/>
</LinearLayout>
```

Input Controls (II)

Control Type	Description	Related Classes
Button	A push-button that can be pressed, or clicked, by the user to perform an action.	`Button`
Text field	An editable text field. You can use the AutoCompleteTextView widget to create a text entry widget that provides auto-complete suggestions	`EditText`, `AutoCompleteTextView`
Checkbox	An on/off switch that can be toggled by the user. You should use checkboxes when presenting users with a group of selectable options that are not mutually exclusive.	`CheckBox`
Radio button	Similar to checkboxes, except that only one option can be selected in the group.	`RadioGroup`, `RadioButton`
Toggle button	An on/off button with a light indicator.	`ToggleButton`
Spinner	A drop-down list that allows users to select one value from a set.	`Spinner`
Pickers	A dialog for users to select a single value for a set by using up/down buttons or via a swipe gesture. Use a DatePickercode> widget to enter the values for the date (month, day, year) or a TimePicker widget to enter the values for a time (hour, minute, AM/PM), which will be formatted automatically for the user's locale.	`DatePicker`, `TimePicker`

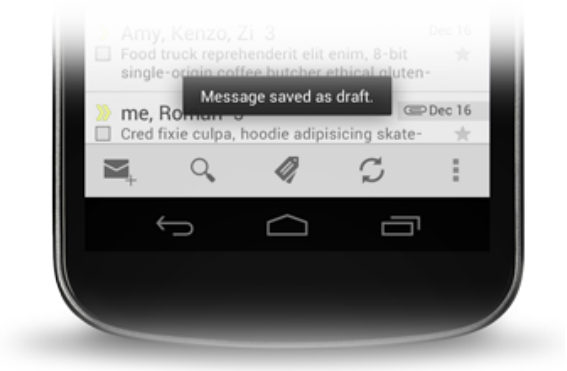
Check: `https://developer.android.com/guide/topics/ui/controls.html`

Toast

- A toast provides simple feedback about an operation in a small popup.
- It only fills the amount of space required for the message and the current activity remains visible and interactive.
  - For example, navigating away from an email before you send it triggers a "Message saved as draft-" toast to let you know that you can continue editing later.
- Toasts automatically disappear after a timeout.

```
Context context = getApplicationContext();
CharSequence text = "Message saved as draft";
int duration = Toast.LENGTH_SHORT;

Toast toast = Toast.makeText(context, text, duration);
toast.show();
```

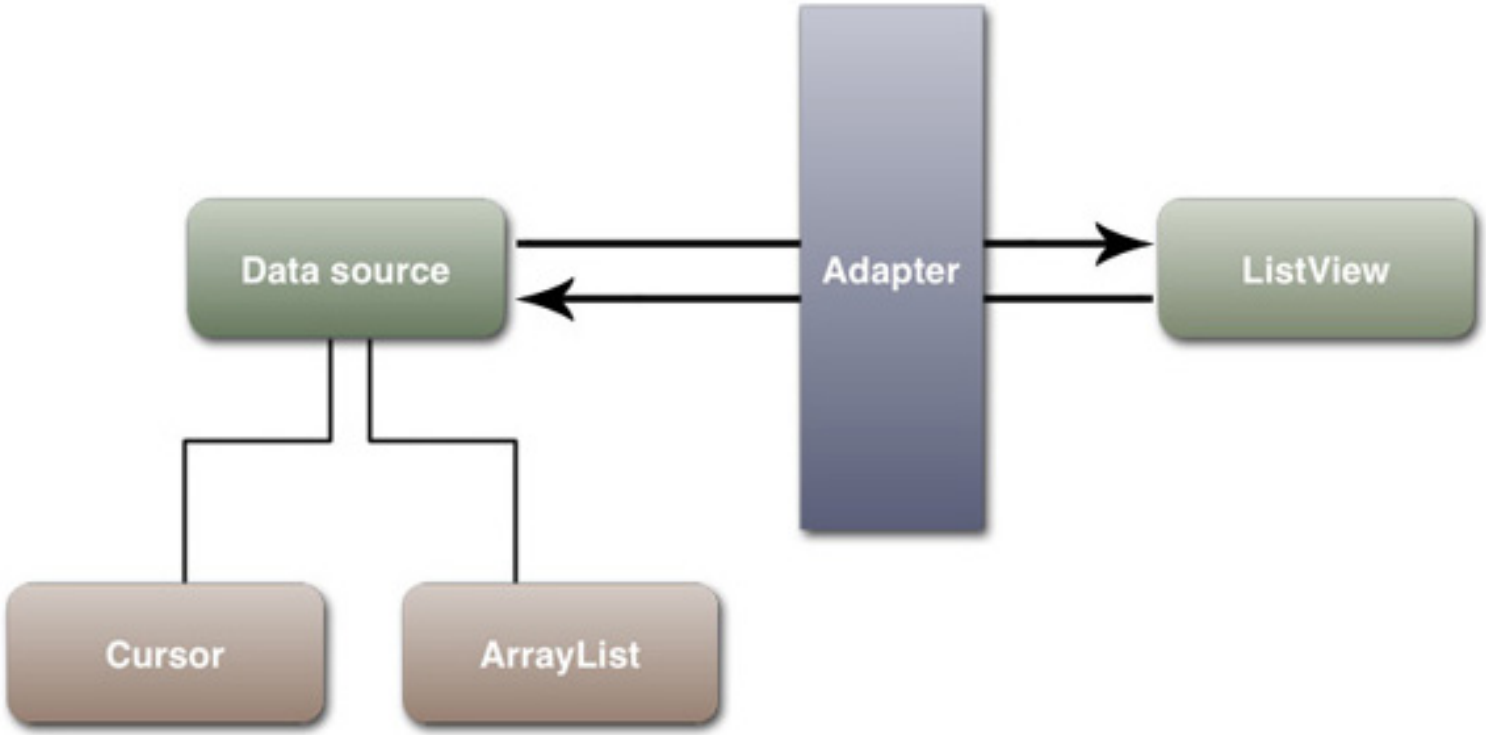


Check: `TP4_02.zip`

AdapterView

Populate an `AdapterView` (I)

- You can populate an `AdapterView` such as `ListView` or `GridView` by binding the `AdapterView` instance to an `Adapter`, which retrieves data from an external source and creates a `View` that represents each data entry.



Populate an AdapterView (II)

- ArrayAdapter
  - Use this adapter when your data source is an array. By default, ArrayAdapter creates a view for each array item by calling toString() on each item and placing the contents in a TextView.

```
 ArrayList<String> dataSource;
 ArrayAdapter<String> adapter;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      ListView lv = (ListView) findViewById (R.id.listView);
       dataSource = new ArrayList<String>();
      String[] students = getResources().getStringArray(R.array.students);
      Collections.addAll(datasource, students);
      adapter = new ArrayAdapter<String>(this,android.R.layout.simple_list_item_1,
            dataSource);
      lv.setAdapter(adapter);
}
```

Populate an `AdapterView` (III)

- Whenever the data source is changed, the `notifyDataSetChanged()` must be invoked.
  - Notifies the adapter that the underlying data source has been changed and any View reflecting the data set should refresh itself.

```
dataSource.add(str);
adapter.notifyDataSetChanged();
```